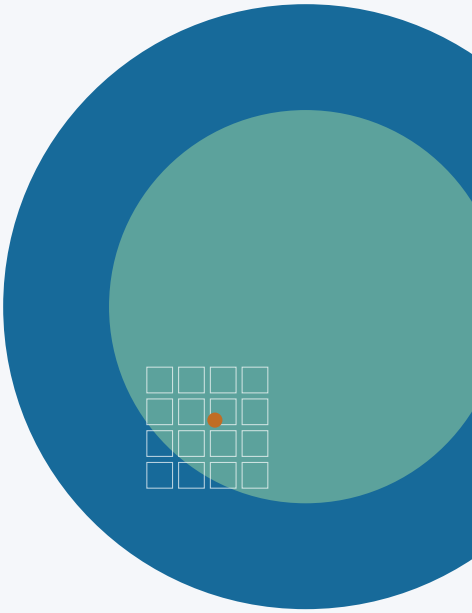
はじめての Bend 2 から、実際に使える Bend 2 へ



マインスイーパーで学ぶ

Bend 2 を使いこなす

標準ライブラリ・入出力・GUI・証明を、自分の道具にする

本文・演習・Bend ソース・独立テスト付き

『はじめての Bend2』から続く実践編

本書の読み方

本書は『ライフゲームで学ぶはじめての Bend2』の続編です。『使いこなす』とは、あらゆる API を暗記することではありません。必要な処理から適切な機能を探し、型と失敗条件を確かめ、実装と検証を自分で進められることです。前巻で学んだ関数、パターンマッチ、配列、アフィン性、`do` の基本から始め、**標準ライブラリの使い分け、入出力、GUI、形式的証明**へ進みます。

目標は、難しいテクニックを暗記することではありません。**Bend 2 で何ができるかを把握し、引数・返り値・所有権・失敗時の挙動を理解して、必要な機能を自分で使えること**です。マインスイーパーはそのための共通の実習題材ですが、ゲームに不要な機能も独立した例で扱います。

要点

各例は「**結果を予想する** → **実行する** → **解説を読む** → **少し改造する**」という順序で進めてください。例を読んだときの疑問が、次の型や仕様の説明につながるようにしています。

教材の使い方

配布 ZIP には PDF、実習用のexamples/、使い始めるための案内が含まれます。ZIP を展開し、そのフォルダーで `bend examples/01_set.bend` のように実行します。リポジトリのクローンは必要ありません。

演習の参考実装や追加の検証資料は、必要なときに 参考コードのリポジトリから参照できます。

学ぶ内容の見取り図

Part	主な学習事項
I	標準ライブラリ、List・Array・Map・Set、失敗を表す型
II	IO、ファイル、引数、乱数、並行処理、通信
III	画像、GUI イベント、マインスイーパーの状態と座標
IV	仕様の定義、law、帰納法、不変条件、証明の範囲
V	並列計算、GPU、外部との連携
VI	マインスイーパーへの統合と運ゲー無し盤面の発展課題

新しく出てくる用語

入門編の基本用語に加えて、次の言葉をよく使います。最初からすべて覚える必要はありません。

用語	この本での意味
標準ライブラリ	言語に用意されたデータ構造や関数の集まり。Bend 2 では `Base` に含まれます。
ハンドル	ファイルや通信など、外部の資源を操作するための値です。使い終えたら閉じる必要があります。
純粋関数	同じ入力に同じ値を返し、画面やファイルなどの外部状態に作用しない処理です。
結果型	成功・失敗や値の有無を、`Maybe` や `Result` などの型で表したものです。
不変条件	ある操作の前後を通じて保たれるべき性質です。
公開情報	マインスイーパーでプレイヤーに見えている数字や状態です。隠れた地雷配置とは区別します。
推論規則	公開情報から確実に言えることを導くための規則です。推測とは異なります。

要点

「型検査に成功した」「実際に動かした」「法則を形式的に証明した」は別の確認です。とくに GUI はデスクトップ環境を必要とするため、型検査だけで画面上の動作まで確認できたとは限りません。

Contents

本書の読み方 ii

新しく出てくる用語 iii

I 標準ライブラリとデータ構造 1

1 Base を調べる 2

2 数値、文字、比較を正しく選ぶ 3

2.1 文字列の前処理と安全な変換 3

3 List と Array の詳しい使い方 5

3.1 List.filter と foldl を組み合わせる 5

3.2 Array.new の引数は長さではない 6

3.3 Array.clone と Array.fork は同じなのか 6

3.4 どの操作が必要かから選ぶ 7

4 Map と Set を使う 8

4.1 「マスに何かを覚えておく」には何が必要か 8

4.2 まず Set で、旗の有無を表す 8

4.3 同じ操作を Map で考える 9

4.4 重複、削除、キーの一覧 10

4.5 使う構造を選ぶ 10

5 失敗、選択肢、依存する値 11

5.1 0 を返せば十分だろうか 11

5.2 失敗の理由を知りたいときは Result 11

5.3 複数の値を返すことと、二種類の結果とは違う 12

5.4 Either を使って「どちらか一方」を表す 12

5.5 Kind と数量指定は何を制限しているのか 13

II 入出力と並行処理 **14**

6 IO を読む **15**
6.1 「ゲームの規則」と「画面に出す」は別の仕事 15
6.2 どうして do の中で順番を書くのか 15
6.3 失敗し得る IO には二重の結果がある 15
6.4 何を IO にするべきか 16
6.5 IO の中で取り出し、純粋関数で加工する 16
6.6 IO と Result は異なる役割を持つ 16
6.7 保存の成功と、保存処理の終了は違う 16

7 File とリソースの受け渡し **18**
7.1 保存するには、どんな順序が必要だろうか 18
7.2 ファイルを開き、書き込んで閉じる 18
7.3 成功と失敗を型で追う 19
7.4 読み戻せたら保存を確認できる 19
7.5 マインスイーパーの保存に応用する 20

8 引数、時刻、乱数、プロセス **21**
8.1 環境変数は常に存在するとは限らない 21
8.2 乱数の取得と失敗の伝播 22
8.3 外部プログラムを起動する 22

9 IO.fork、Chan とタイムアウト **24**
9.1 「二つの仕事を始める」と「結果を待つ」 24
9.2 Chan は何を受け渡すのか 24
9.3 送る側と受け取る側を一つの例で確認する 25
9.4 時間内に終わらなかったら 25
9.5 いつ使うべきか 26

10 TCP、UDP と外部環境 **27**
10.1 通信はファイルと似ているだろうか 27
10.2 TCP は接続を作ってから使う 27
10.3 UDP は何が違うのか 27
10.4 接続できなかった場合の最小例 28
10.5 安全に試すには 28

III Image・App・GUI **29**

11 Image と四分木 **30**
11.1 画面を描くには、どの単位で考えるのか 30
11.2 マインスイーパーならどう描くか 30

12 App とイベントの処理 **31**
12.1 画面が変わるまでを三つに分ける 31
12.2 App の view と tick 31
12.3 本物のゲームでは、状態の更新を純粋関数にする 32

13 マインスイーパーの状態 **33**
13.1 マスには二種類の情報がある 33
13.2 周囲の地雷数を数える 34
13.3 操作を定義する順序 35
13.4 16 マスの状態を Array に格納する 35

14 表示と入力をテストする **36**
14.1 クリックの位置を盤面の番号へ変換する 36
14.2 テストを分けて考える 37

IV 仕様駆動開発と形式的証明 **37**

15 law を先に書く **38**
15.1 まず「こう動くはずだ」を書き出す 38
15.2 見かけ上もっともらしい仕様の落とし穴 39
15.3 証明済みと、動作確認済みは違う 39
15.4 要求を別ファイルへ分けてから証明する 39

16 等式、帰納法、補題と書き換え **41**
16.1 三つの状態の証明から始める 41
16.2 任意の Bool について法則を証明する 41
16.3 再帰と帰納法の対応を見る 42
16.4 等式を使い回すための操作 42
16.5 証明の信頼範囲 42

17 盤面の不変条件 **43**
17.1 一つのマスから盤面全体へ 43
17.2 公開したマスは閉じない 43
17.3 0 の連鎖公開と表示する数字 43

17.4 証明をどこから始めるか 44
17.5 不変条件を実際の盤面に結び付ける 44
17.6 クリックの勝敗を状態として表す 44
17.7 証明と独立した実行確認を並べる 45

18 証明の範囲と信頼境界 **46**
18.1 一つの法則が通れば全部正しいだろうか 46
18.2 検査を三つの段階に分ける 46
18.3 外部コードや unsafe があるとき 46
18.4 実際にどのように報告するか 47

V 並列実行とその他の応用 **47**

19 CPU、GPU、Array.atomic **48**
19.1 計算を独立した二つの枝へ分ける 48
19.2 GPU 指定の感嘆符は高速化の約束ではない 49
19.3 IO.fork は別の仕事を待ちながら進める 49
19.4 配列を独立に複製するか、共有するか 49

20 C・JavaScript と外部連携 **51**
20.1 作成するファイルの種類を選ぶ 51
20.2 Bend のモジュールとパッケージ 51
20.3 生成したコードを動かすまでが検証 52
20.4 パッケージを公開する前に確かめる 52
20.5 外部実装と形式的証明の境界 52
20.6 Audio は開いてから書く 52

VI マインスイーパー統合と発展 **53**

21 一人で遊べるアプリにする **54**
21.1 ゲームに必要な状態を先に決める 54
21.2 クリックすると何が起きるかを表にする 54
21.3 一手目を安全にする 55
21.4 数字が 0 ならどこまで開いてよいか 55
21.5 勝敗は「旗の数」で決めない 56
21.6 GUI からゲームの規則へ入力を渡す 56
21.7 時間・リセット・保存は境界の仕事 56
21.8 部品から完成へ進むための検査順 56

22 発展：運ゲー無しマインスイーパー **58**

22.1 なぜ通常の盤面では推測が必要になるのか 58

22.2 最小の推論規則を Bend 2 で書く 58

22.3 複数の数字から推論する 60

22.4 公開情報と真の地雷配置を分離する 60

22.5 候補生成と検証を繰り返す 60

22.6 小盤面用の参考実装と検証 61

22.7 「運ゲー無し」の保証範囲 61

23 API 索引と次の学習 **62**

PART I

標準ライブラリとデータ構造

この Part の目次

1 Base を調べる 2
2 数値、文字、比較を正しく選ぶ 3
3 List と Array の詳しい使い方 5
4 Map と Set を使う 8
5 失敗、選択肢、依存する値 11

Chapter 1

Base を調べる

> API の宣言、型、数量指定、結果を返す形を読み取る。

入門編では `import Base` を使いました。ここでは、`bend base` と `bend base Array` を使い、提供されている関数を自分で調べます。最新情報はhttps://github.com/bendlang/bend/blob/main/bend2/base.bendも参照してください。

例えば `Set.has` の戻り値は単なる `Bool` ではなく `Set() & Bool` です。「所属を調べる際に集合が消費され、再利用するには返された集合を受け取る」という設計を、この型だけから読み取れます。`Map.get` も元の辞書と値を組で返します。型名、消費する引数、戻ってくる所有物を最初に確認する習慣が重要です。

```
bend base
bend base Set
bend base Map
bend guide
```

すべての定義が通常のユーザー向け API とは限りません。`.go` や `.fin` など実装上のヘルパーも混在しています。最初は外側の公開操作を学び、実装を読む必要が生じたときに内部関数へ進みます。

ハンズオン／確認問題

`Array.get`、`Set.has`、`File.open` を調べ、各関数について「入力」「結果」「失敗の有無」を記録してみましょう。

Chapter 2

数値、文字、比較を正しく選ぶ

整数と浮動小数点、文字列、変換時の失敗を整理する。

Nat は非負の自然数、U32 は固定幅の非負整数、F32 は近似的な浮動小数点数です。サイズの上限、演算結果、オーバーフロー、丸めが同じではありません。型が違う数をそのまま加算できるとは限らず、U32.to_nat など明示的な変換を使います。

String には split、join、trim、starts_with、to_upper などがあり、Char には文字に関する操作があります。単純な文字列の切り分けと、符号化や文字境界の意味を伴う操作は区別します。文字列を整数として読む U32.read は失敗し得るため、返り値の Maybe を確認します。

2.1 文字列の前処理と安全な変換

ユーザーが入力した文字列には、前後に空白が付いていることがあります。最初に String.trim で前後の空白を取り除き、次に U32.read で非負整数に変換する例を動かしましょう。

```
import Base

# String.read-like conversion can fail. Whitespace is removed first.
def parsed_number(value: Maybe<&2, U32>) -> U32:
  match value:
    case Some{n}:
      n
    case None{}:
      0

def main() -> U32:
  cleaned = String.trim(" 42 ")
  parsed_number(U32.read(cleaned))
```

Listing 2.1: 空白を除去して文字列を数値に変換する

U32.read は Maybe 型を返します。成功は Some、失敗は None です。このサンプルでは挙動を確認しやすいよう、失敗時に 0 を返しますが、**実際のアプリでは「失敗した 0」と「入力された 0」を混同しない設計**にしましょう。変換結果を Maybe のまま呼び出し元へ渡す方が明確です。

String.split は区切り文字を Char 一文字で受け取り、文字列のリストを返します。String.join はその逆方向の処理です。split と join の名前だけを覚えるのではなく、**区切り文字の型と、空入力のときの結果**を API と実行例で確認してください。保存ファイルの 1 行を String.split で分解する場合も、項目数と数値変換の失敗を調べる必要があります。

Cmp は大小の比較結果を表し、Bool は真偽を表します。「大小の三種類」と「条件式の二種類」を

使い分けると、ソートや検索の API を読みやすくなります。

ハンズオン／確認問題

文字列`"42"` と`"x"` を数値として読む例を作り、成功・失敗の結果の型と中身を説明してみましょう。

Chapter 3

List と Array の詳しい使い方

構造の違いと、関数の選択が所有権に与える影響を理解する。

List は Nil と Con からなる再帰的な鎖です。map、filter、foldl、foldr、sort などが用意されています。変換したリストを作るとき、入力要素の型が Type か Data かによって、要素を何度も使えるかが変わります。

```
import Base

# A linked list, and its length computed by the Base library.
def main() -> Nat:
  xs = List.replicate(U32, 3n, 7)
  List.length(&2, U32, xs)
```

Listing 3.1: リストを作り要素数を求める

このプログラムは List.replicate で値を 3 個並べたリストを作り、その長さを返します。List.length の引数には、数量と要素型が必要です。先頭への追加には <> という記法も使えます。

3.1 List.filter と foldl を組み合わせる

List.filter は条件を満たす要素だけを残します。List.foldl は先頭から値を集計し、最後の一つの値にまとめます。「抽出」と「集計」は別の操作です。

```
import Base

# List.range returns List<&2, Nat>. Filter keeps that quantity.
def small(n: Nat) -> Bool:
  Nat.is_lt(n, 3n)

def sum(acc: Nat, n: Nat) -> Nat:
  Nat.add(acc, n)

def main() -> Nat:
  xs = List.range(4n)
  ys = List.filter(~Nat, ~small, xs)
  List.foldl(~&2, ~Nat, ~Nat, ~sum, ys, 0n)
```

Listing 3.2: 小さい整数だけを抽出して合計する

List.range(4n) は 0 から 3 までのリストを作り、List.filter で 3 未満の値を残します。その合計は 3n です。foldl は「現在までの合計」と「次の要素」を受け取る関数を引数にします。この二つは

関数を受け取る**高階関数**です。今回必要なのは「選ぶ」と「集計する」ことなので、それぞれ標準の関数を使う方が、自分で状態管理の再帰を追加するよりも計算の意図が明快になります。`foldl` の最後の `0n` は初期値です。ここを 5 に変えると結果も 5 だけ増えます。

`List.filter` は `List<&2, A>` を扱う一方、`List.map` は `List<A>` を受け取るなど、同じ `List` に見えても数量指定が異なる API があります。呼び出せないときは、引数の個数だけでなく**型の数量指定**まで比較しましょう。型の制約に合わないリストを強引に再解釈するのではなく、選ぶ関数やデータの構造を見直します。

`Array` はインデックスを使う構造です。`get` と `set` のほか、`swap`、`clone`、`fork`、`join`、`to_list`、`map` などがあります。「読むだけ」のつもりでも所有権付き配列を消費するので、返された配列を引き継ぎます。`clone` は独立した内容を複製し、`fork` は実体を共有するハンドルを作ります。使い方や費用は異なります。

3.2 Array.new の引数は長さではない

`Array.new` の第二引数は「作る要素数」そのものではなく、二分木状の配列の深さです。深さを d とすると要素数は 2^d です。たとえば `Array.new(U32, 3n, 0)` は 8 要素を作ります。

```
import Base

# Array.new(d) produces an array of 2^d elements.
def read_and_sum(r: Array<U32> & U32) -> U32:
  (a, x) = r
  x

def main() -> U32:
  a = Array.new(U32, 3n, 0)
  b = Array.set(U32, a, 2, 17)
  c = Array.set(U32, b, 7, 25)
  x = read_and_sum(Array.get(U32, c, 2))
  x
```

Listing 3.3: 8 要素の配列の 2 番目と 7 番目を書き換える

`Array.set` は更新した配列を返します。以前の配列をそのまま後続で使うのではなく、返された配列を引き継ぎます。`Array.get` は配列と読み出した値の組を返します。例では配列を再使用しないため捨てていますが、次の読み出しや更新を行う場合は受け取ってください。インデックスが範囲外の場合の扱いも、公開 API の実装を確認しましょう。

3.3 Array.clone と Array.fork は同じなのか

ここで「配列を二つの処理に渡したい」という場面を考えます。片方で値を書き換えても、もう片方に影響しない方がよい場合は、`Array.clone` で**独立した複製**を作ります。

```
import Base

# Array.clone returns two independent copies.
def read_value(result: Array<U32> & U32) -> U32:
  (unused_array, value) = result
  value

def compare_clones(pair: Array<U32> & Array<U32>) -> U32:
  (left, right) = pair
  changed = Array.set(U32, left, 0, 17)
```

```
  modified = read_value(Array.get(U32, changed, 0))
  original = read_value(Array.get(U32, right, 0))
  (modified + original : U32)

def main() -> U32:
  source = Array.new(U32, 2n, 5)
  compare_clones(Array.clone(U32, source))
```

Listing 3.4: 独立した二つの配列を作る

両方の配列の初期値は 5 ですが、片方の 0 番目だけを 17 に変更します。もう片方の 0 番目は 5 のままなので、最後の合計は 22 です。結果を予想した後、コード内の更新値を変えて違いを確かめてください。

一方、`Array.fork` は配列の実体を共有した複数のハンドルを作ります。独立したコピーを作る `clone` とは異なり、**一方の変更がもう一方から見える**ことがあります。同じ場所へ並行して書き込む際には、原子的操作と実行順序の問題もあります。ここではまず `clone` を使い、`fork` と `join` の制約は並列計算の章で確認しましょう。

ハンズオン／確認問題

同じ数値列を `List` と `Array` の両方で表し、要素の変更、走査、複製が必要な場合の選び方を比較してみましょう。

3.4 どの操作が必要かから選ぶ

「列を条件で絞って合計する」ときは、先ほどの `17_list_pipeline.bend` のように `List.filter` と `List.foldl` が自然です。この二つは関数を引数に受け取る**高階関数**です。処理を分けることで、どの要素を選び、どのように集計するかが明確になります。自作の再帰が必要になる場面もありますが、標準の機能で素直に書ける場合はそれを使いましょう。

反対に「盤面の 7 番のマスを書き換える」なら、番号を指定する `Array.set` が適しています。「盤面を検査した後も保持する」なら、`Array.clone` と `Array.to_list` を組み合わせる方法が考えられます。何をしたいかを先に定めれば、必要のない抽象化やデータ構造を増やさずに済みます。

ここまでの例では、`17_list_pipeline.bend` の集計結果は 3、`18_array_mutations.bend` の読み出し結果は 17、`30_array_clone.bend` の合計は 22 でした。この三つの数値を暗記する必要はありません。**元の値、各操作が変えた値、最後に読み出す値**を順に追えば導けます。

ハンズオン／確認問題

`17_list_pipeline.bend` で `small` の条件を「2 未満」にして結果を予想しましょう。次に `foldl` の初期値を `5n` に変更して再度確かめます。また 4×4 の盤面 16 マスを `Array.new` で作るにはどの深さが必要でしょうか。理由とともに説明しましょう。

Chapter 4

Map と Set を使う

どのデータを記録したいかから、Map と Set を選び、更新後の所有権まで扱う。

4.1 「マスに何かを覚えておく」には何が必要か

前巻のライフゲームでは、各マスの状態を配列に入れました。マインスイーパーでも配列は使えますが、例えば**旗を立てた場所だけを記録する**なら、すべてのマスに値を用意する必要はありません。座標"2,3" が旗の集合に含まれるかだけ分かれば十分です。

一方、**マスごとの数字や戦績を対応付ける**ときには、場所と値の組を覚えたいはずです。前者には Set、後者には Map が向いています。どちらも Base にあるため、独自の探索用データ構造を作る前に使い方を確認します。Bend 2 の両方の構造は文字列をキーとして使います。

4.2 まず Set で、旗の有無を表す

```
import Base

# Set contains String keys; membership returns the set as well.
def membership(r: Set() & Bool) -> U32:
  (again, exists) = r
  match exists:
    case True{}:
      1
    case False{}:
      0

def main() -> U32:
  s0 = Set.new()
  s1 = Set.add(s0, "2,3")
  membership(Set.has(s1, "2,3"))
```

Listing 4.1: 集合へ座標を追加して調べる

この例では"2,3" を追加し、Set.has が真を返すことを確かめています。実行結果は 1 です。読者がまず注目したいのは「どうして所属判定の結果が単なる Bool ではないのか」という点です。

Set.has の結果は Set() & Bool です。**調べた後も集合を引き続き使えるよう、集合自体が返ってくる**のです。コード内で返り値を (again, exists) に分け、今回使わない again を捨てています。続けて旗を更新したいなら、その返された集合を次の操作に渡します。

次の例は、調べた結果に応じて追加と削除を選びます。

```
import Base

# Keyed squares: Set operations are affine, so the new set is passed on.
def from_membership(r: Set() & Bool, key: String) -> Set():
  (s, present) = r
  match present:
    case True{}:
      Set.del(s, key)
    case False{}:
      Set.add(s, key)

def toggle_mark(s: Set(), +key: String) -> Set():
  from_membership(Set.has(s, key), key)

def main() -> Nat:
  s0 = Set.new()
  s1 = toggle_mark(s0, "2,3")
  s2 = toggle_mark(s1, "2,3")
  Set.size(s2)
```

Listing 4.2: 同じ場所の旗を二回切り替える

一回目の操作で"2,3" に旗が付き、二回目で取り除かれるため、最後の集合の要素数は 0n になります。0n は Nat 型の 0 です。結果を実行してから追うのではなく、**一行ごとにどの集合が次の行へ渡されるか**を紙に書いてみるとアフィン性が理解しやすくなります。

4.3 同じ操作を Map で考える

Set に記録できるのはキーの存在です。Map ではキーに値を割り当てます。例えばキー"wins" に値 7 を保存できます。

```
import Base

# Map.get returns a rebuilt map and its value.
def take_value(r: Map<&2, U32> & U32) -> U32:
  (again, value) = r
  value

def main() -> U32:
  m0 = Map.new(&2, U32)
  m1 = Map.set(&2, U32, m0, "wins", 7)
  take_value(Map.get(U32, 0, m1, "wins"))
```

Listing 4.3: 辞書に値を保存して取り出す

Map.new では値の型と数量を指定します。Map.set は更新された辞書を返し、Map.get は**辞書と取り出した値の組**を返します。この例の実行結果は 7 です。

ここで気になるのは、辞書に存在しないキーを読んだときの振る舞いでしょう。Map.get には**既定値**を渡す引数があり、見つからなければその値が返ります。例えば既定値が 0 なら、戻り値の 0 が「登録済みの 0」なのか「未登録」なのか、この結果だけでは分かりません。Map.has で存在を確かめるか、値自体を Maybe 等にする設計を選びます。この違いは、保存データの欠落をエラーにするかどうかに直結します。

4.4 重複、削除、キーの一覧

```
import Base

# Set.add discards duplicates; Map.keys returns a list of String keys.
def main() -> Nat:
  s0 = Set.new()
  s1 = Set.add(s0, "0,0")
  s2 = Set.add(s1, "0,0")
  s3 = Set.add(s2, "1,1")
  s4 = Set.del(s3, "0,0")
  m0 = Map.new(&2, U32)
  m1 = Map.set(&2, U32, m0, "wins", 3)
  m2 = Map.set(&2, U32, m1, "games", 7)
  keys = Map.keys(&2, U32, m2)
  a = Set.size(s4)
  b = List.length(&2, String, keys)
  Nat.add(a, b)
```

Listing 4.4: 重複を除く Set と、キーを列挙する Map

同じキーを集合へ二度追加しても、要素数は増えません。一方、辞書の `Map.keys` は、格納されたキーのリストを返します。例の最後の結果は、集合の要素数 1 と辞書のキー数 2 を合わせた `3n` です。

ただし、`Map.keys` の返す順番が登録順だとは仮定しないでください。順番に意味がある画面を作る場合は、別のリストで順番を管理するか、明示的に並べ替えます。

4.5 使う構造を選ぶ

今回の題材では、地雷の位置と旗の位置は `Set`、名前付き設定や戦績は `Map`、盤面全体のマスの並びは `Array` という選び方が考えられます。必ずこの組合せにしなければならないわけではありません。**何を検索したいか、更新した後どの値が戻るか**を基準に選べば、構造の名前だけを覚えるより応用が利きます。

ハンズオン／確認問題

`16_map_set_api.bend` で削除の行を一つ外した場合、最後の値はいくつになりますか。実行前に予想し、変更後の値と一致するか確かめましょう。

Chapter 5

失敗、選択肢、依存する値

「値がない」と「操作を受け付けない」を区別し、型を選べるようになる。

5.1　0 を返せば十分だろうか

座標を調べる関数が 0 を返したとします。これは「周囲の地雷数が 0」なのか、「その座標が見つからなかった」のか、関数の外からは判断できません。値そのものと、値を得られなかったという状況を別々に表す必要があります。

前巻で学んだ `Maybe` は、`Some{value}` と `None{}` を区別します。存在しなかった理由までは必要ない場合に便利です。例えば数値として読めない文字列を `U32.read` へ渡すと `None{}` になり得ます。既出の `15_string_parse.bend` をもう一度見て、成功と失敗でどの分岐を通るか確認しましょう。

5.2　失敗の理由を知りたいときは Result

マインスイーパーでは「すでに開いたマスには旗を付けられない」という操作を考えます。失敗したことだけでなく、なぜ操作できないかを伝えたいとき、`Result` を使います。

```
import Base

# Reject flagging an already revealed square.
# This is a pure game-rule failure, not an IO error.
type Cell is Data:
  Hidden{}
  Flag{}
  Open{}

def flag(s: Cell) -> Result<&2, &2, String, Cell>:
  match s:
    case Hidden{}:
      Done{Flag{}}
    case Flag{}:
      Done{Flag{}}
    case Open{}:
      Fail{"an open square cannot be flagged"}

def success(r: Result<&2, &2, String, Cell>) -> U32:
  match r:
    case Done{cell}:
      1
```

```
    case Fail{reason}:
      0

law hidden_can_be_flagged:
  {flag(Hidden{}) == Done{Flag{}} : Result<&2, &2, String, Cell>}

def hidden_can_be_flagged():
  {==}

def main() -> U32:
  a = success(flag(Hidden{}))
  b = success(flag(Open{}))
  (a + b : U32)
```

Listing 5.1: 旗を付ける操作の成功と失敗

`flag` は、閉じたマスなら `Done{Flag{}}` を、開いたマスなら `Fail{...}` を返します。これは**純粋関数で判定したゲームの規則**です。ファイルへの書き込み失敗のような IO エラーとは分けて考えます。コードの `success` は成功なら 1、失敗なら 0 に変えているため、`main` は `1` を返します。

`Result` を読む際は、**成功の値の型**と**失敗の理由の型**を別々に確かめましょう。この例では成功の値は `Cell` で、失敗の理由は `String` です。`Result` の宣言には数量引数も含まれているため、型を記すときは `Result<&2, &2, String, Cell>` のようになります。こうした追加の型引数を適当に省略すると、型検査で失敗することがあります。

5.3 複数の値を返すことと、二種類の結果とは違う

`A & B` は二つの値をまとめた組です。`Map.get` が「辞書と読み出した値」を返すのは、**両方を同時に持っている**からです。一方、`Either` の `Inl` と `Inr` は、**どちらか一方が入っている**ことを表します。`Result` はこの発想を成功・失敗という用途に特化した型です。

`Sigma` はさらに一般的な組で、**最初の値によって次の値の型が決まる**場合に使います。普通の二つの数を返すだけなら依存型にする必要はありません。この段階では「普通の組と、依存する組を区別できる」ことを目標にし、実際の存在命題と依存型は証明の Part で扱います。

5.4 Either を使って「どちらか一方」を表す

「設定ファイルに数値がある場合」と「説明文をそのまま扱う場合」の二種類を一つの値として持ちたいとします。二つが**同時にある**わけではないので、普通の組とは違います。`Either` は左側の `Inl` または右側の `Inr` で、異なる二種類の値を受け取れる型です。

```
import Base

# Either represents one of two choices, rather than two simultaneous values.
def meaning(choice: Either<&2, &2, U32, String>) -> U32:
  match choice:
    case Inl{number}:
      number
    case Inr{text}:
      U32.from_nat(String.length(text))

def main() -> U32:
  meaning(Inl{7})
```

Listing 5.2: Either の二種類を場合分けする

この例では左側に U32、右側に String を入れられます。meaning は Inl なら数値をそのまま返し、Inr なら文字列の長さを U32 に変換して返します。main は Inl{7} を渡すため、結果は 7 です。

ここで Result との違いを整理しましょう。Either の左右には、成功・失敗という意味は最初から付いていません。Result はその区別を明確にしたいときに選びます。型を選ぶ際は、**値が二つ必要なのか、二者択一なのか、失敗の理由を表したいのか**を考えます。

ハンズオン／確認問題

main から右側の Inr に短い文字列を渡し、返り値を予想してから実行しましょう。

5.5 Kind と数量指定は何を制限しているのか

ここまでの Map.get や List.filter では、&1・&2・Type・Data などを見ました。これらは装飾ではありません。ある値を複数回使ってよいか、使った後に戻す必要があるかに関係します。Kind はそうした型の分類を表し、Quant は数量を指定する仕組みです。

最初に覚えるべき手順は三つです。関数の引数の型を読むこと、返り値で所有権が戻るかを見ること、複製が必要なら適切な数量指定を使うことです。すべての種類の数量を暗記するより、Set.has と File.read のような実例で意味を確認すると混乱しません。

ハンズオン／確認問題

「座標を探したが見つからない」「保存が失敗したので理由を知らせたい」「辞書と読み出した値を返したい」それぞれに、Maybe、Result、組のどれが適切かを選び、理由を説明してみましょう。

PART II

入出力と並行処理

この Part の目次

6 IO を読む 15
7 File とリソースの受け渡し 18
8 引数、時刻、乱数、プロセス 21
9 IO.fork、Chan とタイムアウト 24
10 TCP、UDP と外部環境 27

Chapter 6

IO を読む

値を計算する処理と、画面・ファイルなど外界に作用する処理を分ける。

6.1 「ゲームの規則」と「画面に出す」は別の仕事

前章の `flag` は、引数が同じならいつ呼んでも同じ結果を返します。画面がなくても検査できる**純粋関数**です。ところが、画面に文章を表示する、ファイルを書き換える、乱数を取得するといった操作は、プログラムの外に影響したり、外の状態に左右されたりします。このような**副作用を伴う計算**を表すために、`IO<A>` を使います。

`IO.print("hello")` は `IO(Unit)` です。文章を表示する仕事を表し、特に返す値がないことを `Unit` で表現しています。対して `IO.args()` は `IO(List<String>)` です。起動時の引数をリストとして受け取ります。**`IO` の括弧の中身は、作用の結果として得る値の型**です。

6.2 どうして do の中で順番を書くのか

マインスイーパーの設定を保存するなら、まずファイルを開き、次に書き込み、最後に閉じます。順番を入れ替えると意味が変わる処理です。`do IO<Unit>:` は、こうした順序を読みやすく書くためにあります。

すでに使った `08_args.bend` を見直すと、`<-`の左側で得た値を、次の行へ渡しています。これが「外部の処理が返した結果を受け取り、次の処理を続ける」という流れです。`IO.pure` は、外へ作用せず値を `IO` の結果として返すための操作で、後の `IO.fork` の実例にも登場します。

6.3 失敗し得る IO には二重の結果がある

`File.open` や `IO.random_u32` は、成功するとは限りません。そのため `IO(Result<...>)` のように、**外部の処理**と**その成功・失敗**を両方型に書きます。`IO.try` は、結果が `Done` なら値を取り出し、`Fail` なら後続の計算へ進まず失敗を伝えます。

ここで重要なのは、`IO.try` を使えば必ず回復できるわけではないということです。失敗時に別の保存先を試すなど、回復する方針が必要なら、`Result` の値を直接受け取って `match` で分岐します。第 7 章で実際のファイル操作を見ます。

6.4 何を IO にするべきか

旗を付けられるかの判定、周囲の地雷数、勝敗の規則は純粋関数にできます。一方、結果を画面に表示する、設定を保存する、実際の時刻を読む、といった境界の仕事は IO です。両者を分けると、ゲームの規則を `law` で述べたり、自動テストで調べたりできます。

ハンズオン／確認問題

「マスを開いた後に地雷数を調べる」「戦績をファイルへ保存する」「マウスクリックを待つ」の三つを、純粋な部分と IO が必要な部分に分けてみましょう。

6.5 IO の中で取り出し、純粋関数で加工する

入門編で学んだ `do` は、副作用のある操作を順番に書くための構文です。一方、取り出した値の長さを数える操作まで IO にする必要はありません。`08_args.bend` では `IO.args` が起動時の引数を取得し、`List.length` がその長さを計算して、`IO.print` が結果を表示します。効果のある取得と、純粋な加工が並んでいることを確認してください。

6.6 IO と Result は異なる役割を持つ

乱数取得や環境変数の読み取りは実行時の環境に依存します。`11_random_io.bend` は `IO.random_u32` の結果を `IO.try` で受け取り、成功した数値を表示します。対して `19_environment.bend` は `IO.get_env` から得た `Result` を `match` し、`Done` なら文字列長を、`Fail` なら 0 を返します。

失敗を 0 に置き換えることが常に適切なわけではありません。統計上「取得できなかった件数は 0 と数える」場合と、ゲーム保存が失敗してプレイヤーに通知する場合を比べてみましょう。**失敗を上位へ伝える必要があるなら、Result の情報を残す**のが重要です。関数の返り値は `bend base IO.try` などで確認できます。

6.7 保存の成功と、保存処理の終了は違う

`File.write` が返す値には、書き込みの成否だけでなく、次の操作へ渡すファイルハンドルも含まれます。`12_file_write.bend` では、`Done` でも `Fail` でも `File.close` を実行します。`13_file_read.bend` でも同様です。失敗時だけ後始末を飛ばす実装では、アフィンなリソースの取り扱いが不完全になります。

注意

ファイルの例は、同名の `sample_board.txt` が存在しない書き込み可能なディレクトリで順に実行してください。サンプルは既存ファイルを上書きする練習ではありません。型検査が通ることと、実際の保存先に書き込めることも異なります。

ハンズオン／確認問題

`19_environment.bend` の `Fail` で 0 を返す設計と、`Result` のまま失敗を呼び出し側へ伝える設計を比較しましょう。ファイル書き込みに失敗した後も `File.close` が必要な理由を、リソースの受け渡しから説明してください。

Chapter 7

File とリソースの受け渡し

保存と読み戻しを通して、File ハンドルの所有権と失敗処理を理解する。

7.1　保存するには、どんな順序が必要だろうか

マインスイーパーの戦績を次に起動したときも使いたいとします。プログラムが終了しても情報を残すには、ファイルへ書き込む必要があります。

保存には「ファイルを開く → 書き込む → 閉じる」という順序があります。先に閉じてから書くことはできません。また、ファイルを開けなかったときには、書き込みに進んではいけません。この流れがそのまま Bend の型と返り値に表れています。

7.2　ファイルを開き、書き込んで閉じる

```
import Base

# File operations return the handle so it can be used or closed exactly once.
# Run in a writable directory. This writes "sample_board.txt".

def close_after_write(r: File & Result<&1, &1, U32 & String, Unit>) -> IO(Unit):
  (file, status) = r
  match status:
    case Done{value}:
      File.close(file)
    case Fail{error}:
      do IO<Unit>:
        Unit <- File.close(file)
        IO.die(Unit, 1, "Could not save sample_board.txt")

def main() -> IO(Unit):
  do IO<Unit>:
    file: File <- IO.try(File, File.open("sample_board.txt", "w"))
    written: File & Result<&1, &1, U32 & String, Unit> <- File.write(file, "board-size=4\nmines=1,1\n")
    close_after_write(written)
```

Listing 7.1: ファイルへ盤面の設定を書き込む

この例を実行すると、作業フォルダーに `sample_board.txt` が作られます。まず `File.open` で書き込み用に開き、`File.write` で二行の文字列を書きます。最後に `File.close` で閉じます。

ここで**ファイルハンドル（file handle）**に注目しましょう。ハンドルは「いま開いているファイルを操作するための値」です。Bend 2 では、書き込みへ渡したハンドルを、その後も同じ変数から勝手に使い続けることはできません。`File.write` は**継続して使うハンドルと書き込み結果**を組で返します。成功時も失敗時も、返されたハンドルを受け取り、最後に閉じます。

要点

`File.open` で失敗した場合には、まだ使用可能なハンドルがありません。一方、`File.write` で失敗した場合には、書き込みの結果を受け取ってから**ハンドルを閉じる処理**が必要です。どの時点でリソースが存在しているかを区別しましょう。

7.3 成功と失敗を型で追う

`File.open` は `IO(Result<...,File>)` を返します。外部の操作であることに加え、その操作が失敗し得ることを表す型です。この例では `IO.try` で成功したファイルを取り出し、開けなかった場合は失敗を伝えます。

書き込みを行う `File.write` はハンドルと `Result` を返すので、`Done` と `Fail` を `match` で分けます。失敗したら後の処理へ進まないというだけでなく、**閉じ忘れを防ぐ**ように両方の分岐で `File.close` しています。

7.4 読み戻せたら保存を確認できる

ファイルに文字列を書いたとしても、保存した内容を本当に使えるかはまだ分かりません。今度は読み込みを試します。

```
import Base

# Reads the file written by 12_file_write.bend.
# Every branch closes the affine File handle.

def close_after_read(r: File & Result<&1, &1, U32 & String, String>) -> IO(String):
  (file, status) = r
  match status:
    case Done{contents}:
      do IO<String>:
        Unit <- File.close(file)
        return contents
    case Fail{error}:
      do IO<String>:
        Unit <- File.close(file)
        IO.die(String, 1, "Could not read sample_board.txt")

def main() -> IO(Unit):
  do IO<Unit>:
    file: File <- IO.try(File, File.open("sample_board.txt", "r"))
    result: File & Result<&1, &1, U32 & String, String> <- File.read(file, 4096)
    contents: String <- close_after_read(result)
    IO.print(contents)
```

Listing 7.2: 保存した文字列を読み戻す

まず `12_file_write.bend` を実行し、次に `13_file_read.bend` を実行してみましょう。読み込んだ文字

列が保存した二行と同じであることを確認します。このように**書き込みから読み込みまで一組として確かめること**をラウンドトリップテストと呼びます。

`File.read(file,4096)` の数値は、一度に読み取る最大量です。大きなファイルを読んだときに、一度の呼び出しで必ず全体が戻ると思ってはいけません。終端まで繰り返す必要がある場合もあります。また、`File.read_bytes` と `File.write_bytes` は、文字列ではなくバイト列を扱う API です。

7.5 マインスイーパーの保存に応用する

実際の戦績を保存するときは、盤面サイズ、地雷数、勝利数などを別の項目にします。読み込んだ文字列は外部から渡されたデータです。値が欠けている、数字として読めない、範囲から外れている場合には、ゲームの内部状態として採用せず、`Maybe` や `Result` で失敗を表します。読み込みに成功したことと、**内容が正しいこと**は別です。

ハンズオン／確認問題

書き込み例の`"board-size=4"` を別の値に変更して保存し、読み込み例で表示が変わるか確認しましょう。次に、ファイルを削除した状態で読み込み例を実行するとどうなるか、実行前に予想してみましょう。

Chapter 8

引数、時刻、乱数、プロセス

外部情報を受け取る方法と、再現できるテストの作り方を学ぶ。

`IO.args` は起動時の引数をリストとして受け取ります。設定ファイルの位置や盤面のサイズを指定する例で使えます。`IO.get_env` は環境変数を読みます。

```
import Base

# IO.args returns a list including the executable or source path.
def main() -> IO(Unit):
  do IO<Unit>:
    xs: List<String> <- IO.args()
    IO.print(Nat.show(List.length(&1, String, xs)))
```

Listing 8.1: 引数の個数を表示する

8.1 環境変数は常に存在するとは限らない

環境変数は OS や起動方法によって変化します。`IO.get_env("PATH")` はパス検索の設定を読む例ですが、必ず値があると仮定してはいけません。

```
import Base

# The environment may not contain the requested name.
def env_length(value: Result<&1, &1, U32 & String, String>) -> Nat:
  match value:
    case Done{s}:
      String.length(s)
    case Fail{problem}:
      On

def main() -> IO(Unit):
  do IO<Unit>:
    result: Result<&1, &1, U32 & String, String> <- IO.get_env("PATH")
    IO.print(Nat.show(env_length(result)))
```

Listing 8.2: 環境変数を読み取り長さを表示する

`IO.get_env` の結果は `Result` です。`Done` の場合は文字列を受け取り、`Fail` の場合は例のように既定動作へ分岐できます。エラー内容が必要なら、`Fail` の値を捨てずに利用します。環境変数の中身をそのままログに出すと、パスワードなどの機密情報を漏らす可能性があります。この例では値そのものではなく長さだけを出力します。

IO.now は現在時刻、IO.sleep は時間待ち、IO.random_u32 は乱数を扱います。乱数を使ったテストは、実際の乱数に依存させず、固定した地雷配置を入力にする方法と分けて実施します。

8.2 乱数の取得と失敗の伝播

IO.random_u32() の型は IO(Result<..., U32>) であり、単に U32 が返るわけではありません。外部機能が失敗する可能性を明示するためです。IO.try を使うと成功した U32 を取り出せますが、失敗したときは後続の IO.print へは進みません。

```
import Base

# The random-number effect may fail, hence the Result inside IO.
def main() -> IO(Unit):
  do IO<Unit>:
    random_value: U32 <- IO.try(U32, IO.random_u32())
    IO.print(U32.show(random_value))
```

Listing 8.3: 乱数を取得して表示する

この例は毎回値が変化するため、数値そのものを固定して比較するテストには不向きです。ソルバーの正しさは固定した盤面でテストし、ランダム盤面の生成はその後に確認しましょう。

8.3 外部プログラムを起動する

Process.run は外部プログラムを起動します。引数のリスト、標準入力へ渡す文字列、受け取り可能な出力の最大量、タイムアウトを指定します。

```
import Base

# Launch a harmless OS program and examine its exit code.
def exit_status(value: U32 & (String & String)) -> U32:
  (code, outputs) = value
  code

def main() -> IO(Unit):
  do IO<Unit>:
    result: U32 & (String & String) <- IO.try(U32 & (String & String), Process.run("/usr/bin/true", Nil{}, "", 1024,
      3000))
    IO.print(U32.show(exit_status(result)))
```

Listing 8.4: 外部プログラムを起動して終了コードを確認する

この例は Unix 系環境の/usr/bin/true を実行します。このコマンドは何も表示せずに成功の終了コード 0 を返します。Windows などでは同じパスは使えないため、実行環境に応じてプログラムを選んでください。

Process.run は IO(Result<...>) 型の結果を返します。プロセスの**起動そのものが失敗した場合**と、**起動したプログラムが 0 以外の終了コードで終わった場合**は違います。前者は Fail で、後者は成功結果内の終了コードで区別します。返り値には標準出力と標準エラー出力も含まれます。ここでは終了コードだけを取り出しますが、利用者に必要な情報を捨ててはいけません。

ハンズオン／確認問題

引数が不足するときは既定の盤面を使うか、理由を表示して終了するか、用途に応じて選んでみましょう。

Chapter 9

IO.fork、Chan とタイムアウト

独立した IO を開始して結果を受け取り、チャネルの終了を扱う。

9.1 「二つの仕事を始める」と「結果を待つ」

画面を表示する処理とは別に、時間のかかる読み込み処理を始めたい場面を考えます。何かを始める処理と、結果が必要になるまで待つ処理を分けられれば、複数の仕事を扱いやすくなります。

Bend 2 の `IO.fork` は IO 計算を開始し、その結果を受け取るための `Chan` を返します。`IO.join` はそのチャネルから結果を待ちます。まずは複雑な並行処理ではなく、確実に 7 が返ってくる小さな仕事で型を確認します。

```
import Base

# fork returns a channel for the result; join waits and closes it.
def main() -> IO(Unit):
  do IO<Unit>:
    chan: Chan(U32) <- IO.fork(U32, IO.pure(U32, 7))
    value: U32 <- IO.join(U32, chan)
    IO.print(U32.show(value))
```

Listing 9.1: IO 計算を開始し結果を受け取る

実行結果は 7 です。`IO.pure(U32,7)` は外部に作用せず 7 を返す IO 計算です。ここで分かるのは、**計算を開始して結果を受け取れること**です。複数の CPU コアを使ったとか、処理時間が短くなったとまでは言えません。

9.2 Chan は何を受け渡すのか

`Chan` は**チャネル（channel）**と呼ばれ、値の送信と受信に使います。`Chan.new` はチャネルを用意し、`Chan.send` は値を送信し、`Chan.recv` は値を受け取ります。`Chan.close` でチャネルを閉じます。

受信時の結果は `Maybe` です。送られた値があれば `Some`、チャネルが閉じられて残りの値もなければ `None` になります。**受信できないときに、存在しない値が適当に返るわけではありません**。送信時も、閉じたチャネルへ送れない場合は `Fail` として扱います。失敗と終了を結果型で読み取るのが、この API の大切な点です。

9.3 送る側と受け取る側を一つの例で確認する

IO.fork では結果を受け取るチャネルが内部的に用意されますが、チャネル自体を直接使うこともできます。

```
import Base

# A one-slot channel: send, receive and close.
# The channel handle is shared by these operations using +chan.

def received_value(result: Maybe<&1, U32>) -> U32:
  match result:
    case Some{value}:
      value
    case None{}:
      0

def exchange(+chan: Chan(U32)) -> IO(Unit):
  do IO<Unit>:
    outcome: Result<&1, &1, U32, Unit> <- Chan.send(U32, chan, 9)
    value: Maybe<&1, U32> <- Chan.recv(U32, chan)
    Unit <- Chan.close(U32, chan)
    IO.print(U32.show(received_value(value)))

def main() -> IO(Unit):
  do IO<Unit>:
    chan: Chan(U32) <- Chan.new(U32, 1)
    exchange(chan)
```

Listing 9.2: チャネルへ 9 を送り、受け取り、閉じる

Chan.new(U32,1) の第二引数はバッファの容量です。この例は容量 1 のチャネルへ 9 を送ります。受信結果は Maybe なので、Some の場合だけ値を取り出します。受信後に Chan.close を呼びます。

この例では +chan を使って、送信、受信、終了処理で同じチャネルを参照しています。これは Chan の特性を使った例です。他のアフィンな値が同様に共有できるとは限りません。また、バッファがない、送受信の相手が現れないといった場合には待機することがあるため、実際のプログラムでは**誰が送るか、誰が受け取るか、いつ閉じるか**を先に設計します。

ハンズオン／確認問題

この例の 9 を別の数値に変えてみましょう。次に、どの行を実行すると受信側が値を得られるか、順序を追って説明してください。

要点

IO.fork と CPU 計算を並列に呼び出す構文は、目的が異なります。前者は IO の開始と結果の受信、後者は独立した計算の並列実行です。速さを考える前に、どの結果をいつ必要とするか整理しましょう。

9.4 時間内に終わらなかったら

IO.within は時間制限内に値が得られたかを Maybe として返します。Some は間に合った結果、None は時間内に結果が得られなかったことを表します。

```
import Base

# A completed result is Some; an expired wait is None.
# The delayed work may continue after the timeout.
def explain(result: Maybe<&1, U32>) -> IO(Unit):
  match result:
    case Some{value}:
      IO.print(U32.show(value))
    case None{}:
      IO.print("Timed out")

def main() -> IO(Unit):
  do IO<Unit>:
    result: Maybe<&1, U32> <- IO.within(U32, 1000, IO.pure(U32, 7))
    explain(result)
```

Listing 9.3: 制限時間内の結果を Maybe で受け取る

ここではすぐ 7 を返す計算を、1000 ミリ秒の制限付きで待ちます。ほとんどの実行環境で 7 が表示されるはずですが、実行スケジューリングの条件によっては結果が変わる可能性もあります。したがって、この例の目的は**必ず 7 が出ることを証明する**のではなく、Some・None を両方処理することです。

ただし**待つ側が時間切れになっても、裏で動く仕事が自動的に停止するとは限りません**。時間が掛かる処理を開始するなら、途中で終了させる方法や、終了後のリソース管理も設計する必要があります。Chan.try_recv や Chan.try_send では Poll の Ready・Wait で、処理できたか、待つ必要があるかを表します。

9.5 いつ使うべきか

小さなマインスイーパーの通常の操作は、まず一つずつ順番に実装する方が理解しやすいでしょう。一方、時間のかかる分析や外部処理を追加する場合には、処理を分ける選択肢があります。並行処理が必要になるまでは、単純な純粋関数と順序どおりの IO で十分です。

ハンズオン／確認問題

21_io_fork.bend が 7 を返す理由を説明してみましょう。次に IO.pure の 7 を別の数値へ変え、IO.join が同じ数値を受け取れるか確かめましょう。

Chapter 10

TCP、UDP と外部環境

通信の開始・送受信・終了と、相手が応答しない場合の扱いを理解する。

10.1　通信はファイルと似ているだろうか

ファイルでは「開く、読み書きする、閉じる」という順序がありました。別のプログラムやコンピュータと通信するときも、**接続する、データを送受信する、接続を閉じる**という順序を考えます。違うのは、相手がいつ応答するか分からないことと、途中で通信が切れることがある点です。

Bend 2 の `Base` には `TCP` と `UDP` の API があります。ネットワーク通信をマインスイーパーへ無理に組み込む必要はありません。この章では、通信する相手がいるときにどの操作を選び、どんな失敗が起こるかを学びます。

10.2　TCP は接続を作ってから使う

TCP では、受け取る側が `TCP.listen` で待ち受け、`TCP.accept` で接続を受け取ります。接続する側は `TCP.connect` を使います。接続して得た `Socket` を `TCP.send` や `TCP.recv` へ渡してデータを送受信します。

`TCP.listen` や `TCP.connect` は `IO(Result<...>)` を返します。利用するアドレスやポートが正しくても、相手が起動していなかったり、アクセスが拒否されたりすることがあります。この場合はハンドルが得られません。接続できた後も、送信と受信には失敗の可能性があり、`Socket` を次の操作へ渡します。

要点

`TCP.send` が成功しても、それは送信内容が相手のアプリケーションで処理されたことまで保証しません。また、一度の `TCP.recv` で送信した文章全体が必ず届くわけではありません。送受信するデータの区切り方を、アプリケーション側で決める必要があります。

10.3　UDP は何が違うのか

UDP では、接続を確立してから使う方法ではなく、送信先を指定したデータグラムをやり取りする API を使います。`UDP.bind` で受信側を用意し、`UDP.send_to` で送信し、`UDP.recv_from` でデータと送

信元の情報を受け取ります。

送信したデータが相手へ必ず届く保証はありません。順番が変わることや同じデータが複数回届くことも考慮します。通信方式の名前だけで選ぶのではなく、「確実に順序どおり受け取りたいか」「一つのデータを独立して送りたいか」を基準に選びます。

10.4 接続できなかった場合の最小例

相手が用意されていない環境でも、**接続の失敗を受け取る**練習はできます。次のコードはローカルのアドレスへ接続を試み、成功した場合はソケットを閉じ、失敗した場合はその旨を表示します。

```
import Base

# Demonstrates connection success/failure, not a working echo server.
def handle(r: Result<&1, &1, U32 & String, Socket>) -> IO(Unit):
  match r:
    case Done{socket}:
      Socket.close(socket)
    case Fail{reason}:
      IO.print("No TCP listener at this address")

def main() -> IO(Unit):
  do IO<Unit>:
    outcome: Result<&1, &1, U32 & String, Socket> <- TCP.connect("127.0.0.1", 1)
    handle(outcome)
```

Listing 10.1: TCP 接続の成功と失敗を区別する

`TCP.connect` が返す `Result` を `match` で処理します。`Done` には利用できる `Socket` が、`Fail` にはエラー情報が入ります。接続に成功した分岐では `Socket.close` を呼びます。これは**失敗する接続の実例**であり、送受信まで行う通信プログラムではありません。接続できる相手が別に起動している場合、実行結果は異なります。

10.5 安全に試すには

最初の通信実験では外部のサービスへ接続せず、同じコンピュータの**ループバック（loopback）**を使いましょう。受信側を開始し、送信側を起動し、届いた内容を表示する順番です。ただし、この節では API の基本的な意味を説明する段階です。TCP・UDP の送受信を一通り動かす独立実習は、続く改訂で追加します。

送受信した値は外部から受け取るデータなので、不正な長さ、不正な文字列、予期しない切断を考慮します。ファイルで学んだ `Result` やハンドル管理が、ここでも役立ちます。

ハンズオン／確認問題

TCP で接続先プログラムが起動していない場合、どの操作が失敗しそうか考えてみましょう。次に、一回の受信で文章全体を受け取れると仮定してはいけない理由を説明してみましょう。

PART III

Image・App・GUI

この Part の目次

11 Image と四分木 30
12 App とイベントの処理 31
13 マインスイーパーの状態 33
14 表示と入力をテストする 36

Chapter 11

Image と四分木

画面を「図形部品」ではなく再帰的な画像として捉え、Pix と Qua を使い分ける。

11.1　画面を描くには、どの単位で考えるのか

普通の GUI ではボタンやテキスト欄などの部品を並べる場面が多いでしょう。しかし Bend 2 の `App` では、まず**現在の状態から画像を作る**方法を理解します。そこで登場するのが `Image` です。

画面全体を一色にしたいなら、すべての画素を一つずつ並べる必要はありません。一色の正方形を `Pix` で表します。一方、左上と右上で色を変えたいなら、四つの領域に分ければよさそうです。これを繰り返し使える構造が `Qua` です。

```
Pix{0x002244}
Qua{Pix{0}, Pix{0xffffff}, Pix{0xffffff}, Pix{0}}
```

最初の例は全体が一色です。次の例は左上と右下が黒、右上と左下が白の四つの領域に分かれています。`Qua` の引数の順番は**左上、右上、左下、右下**です。上下や左右を取り違えると、画像が意図しない配置になります。

11.2　マインスイーパーならどう描くか

盤面は正方形のマスが並ぶ構造なので、マスを画像へ変換し、それらを組み合わせるのが自然です。閉じたマスと開いたマスの色、地雷や旗の模様などを考えます。最初は色だけで状態を区別し、動作を理解してから描画を細かくすると、本来の目的から外れずに進められます。

画像を作ることと、ウィンドウを開くことは別です。`Pix` や `Qua` は画像を表す値を組み立てます。ウィンドウを開いて画面に表示するのは、次章の `App.run` の役割です。

ハンズオン／確認問題

四つのマスのうち左上だけを赤、他を黒にする `Qua` の構成を考えましょう。どの位置にどの `Pix` を置くか、先に紙へ描いてからコードにしてみましょう。

Chapter 12

App とイベントの処理

「今の状態を表示する」と「入力から次の状態を決める」を分離する。

12.1　画面が変わるまでを三つに分ける

マインスイーパーのマスをクリックしたとします。この一回の操作を、次の流れで捉えます。

1. マウスのイベントを受け取る。
2. どのマスが選ばれたかを調べ、ゲームの状態を更新する。
3. 更新後の状態から新しい画像を作る。

「クリックされたら直ちに画面の一部を塗り替える」と考えるよりも、この三つを分けた方が、イベントの処理とゲームの規則を別々に確認できます。

12.2　App の view と tick

App は view と tick という二つの処理を組み合わせます。view は状態から Image を作り、状態と画像の組を返します。tick は List<Event> を受け取り、IO(Maybe<State>) として次の状態を返します。Some で続行し、None で終了します。

```
import Base

# Adapted from the official GUIDE. Requires a graphical desktop to run.
def tick(events: List<Event>, color: U32) -> IO(Maybe<U32>):
  match events:
    case Nil{}:
      IO.pure(Maybe<U32>, Some{color})
    case Con{Close{}, rest}:
      IO.pure(Maybe<U32>, None{})
    case Con{e, rest}:
      tick(rest, (color + 1 : U32))

def main() -> IO(Unit):
  App.run(~U32, ~App{+s => (s, Pix{s}), tick}, "GUI example", 256, 256, 0)
```

Listing 12.1: まず色の変わる最小 GUI を動かす

この例では状態は一つの U32 で、view は Pix{s} を描きます。イベントがないときは同じ色、閉じるイベント Close なら終了、それ以外のイベントなら色の数値を増やします。最初からマインス

イーパーの盤面を入れないのは、**App の役割だけを先に確かめるため**です。

GUI の動作にはデスクトップ環境が必要です。コンパイラの型検査に成功したことと、実際にウィンドウが表示されマウス操作できることを区別してください。

12.3 本物のゲームでは、状態の更新を純粋関数にする

次に色を増やす代わりに、ゲームの操作 `reveal` や `toggle_flag` を呼び出す設計にします。どのマスがクリックされたかは `Event` から得た位置を変換して決めます。ゲーム側の関数へは座標や操作の種類だけを渡せば、GUI を起動しなくても規則をテストできます。

`App.run` は繰り返し状態と表示を扱います。`tick` で受け取るイベントはリストなので、イベントが複数ある場合の処理順序も考える必要があります。最初は一つのイベントを正しく扱い、次に複数のイベントへ進みましょう。

ハンズオン／確認問題

`07_gui_hello.bend` で `Close` を受けたときの返り値と、イベントを受けなかったときの返り値を比較してみましょう。なぜ前者が `None` なのでしょうか。

Chapter 13

マインスイーパーの状態

「地雷の正解」「画面に見えていること」「操作の結果」を混ぜずに設計する。

13.1　マスには二種類の情報がある

プレイヤーがまだ開いていないマスには、地雷があるかもしれません。しかし、**画面には地雷の有無を表示しません**。同じマスについて、「内部にだけある正解」と「プレイヤーが見られる情報」の二種類が存在します。これらを同じ変数として扱うと、ソルバーに正解を渡してしまうような誤りが起こります。

まず、画面に見えるマスの状態だけを作ります。

```
import Base

# The local rules are pure. A later chapter attaches them to a full board.
type Square is Data:
  Hidden{}
  Flag{}
  Revealed{nearby: U32}

def flip_flag(s: Square) -> Square:
  match s:
    case Hidden{}:
      Flag{}
    case Flag{}:
      Hidden{}
    case Revealed{count}:
      Revealed{count}

def is_flag(s: Square) -> U32:
  match s:
    case Flag{}:
      1
    case Hidden{}:
      0
    case Revealed{count}:
      0

def main() -> U32:
  is_flag(flip_flag(Hidden{}))
```

Listing 13.1: 隠れたマス、旗、公開済みマス

この例では Hidden・Flag・Revealed を区別します。さらに Revealed には周囲の地雷数を持たせています。これは画面に見えている状態です。真の地雷位置は別の Set 等で持つ設計にします。

13.2 周囲の地雷数を数える

開いたマスに表示する数字は、周囲 8 か所のうち地雷が置かれている数です。では、左上の隅のマスにも 8 か所の隣接位置があるでしょうか。盤面の外側はマスではないので、実際に調べるのは 3 か所です。この境界条件を先に確認します。

```
import Base

# A 4x4 Minesweeper board; mine positions are String keys "x,y".
# Out-of-bounds neighbor coordinates must NOT wrap around the board.

def member_bit(r: Set() & Bool) -> U32:
  (s, found) = r
  match found:
    case True{}:
      1
    case False{}:
      0

def mine_at(mines: Set(), x: U32, y: U32) -> U32:
  member_bit(Set.has(mines, U32.show(x) ++ "," ++ U32.show(y)))

def guarded(+mines: Set(), x: U32, y: U32, inside_x: Bool, inside_y: Bool) -> U32:
  match inside_x inside_y:
    case True{} True{}:
      mine_at(mines, x, y)
    case _ _:
      0

def adjacent(+mines: Set(), +x: U32, +y: U32) -> U32:
  +left = (x - 1 : U32)
  +right = (x + 1 : U32)
  +up = (y - 1 : U32)
  +down = (y + 1 : U32)
  a = guarded(mines, left, up, U32.is_lt(left, 4), U32.is_lt(up, 4))
  b = guarded(mines, x, up, U32.is_lt(x, 4), U32.is_lt(up, 4))
  c = guarded(mines, right, up, U32.is_lt(right, 4), U32.is_lt(up, 4))
  d = guarded(mines, left, y, U32.is_lt(left, 4), U32.is_lt(y, 4))
  e = guarded(mines, right, y, U32.is_lt(right, 4), U32.is_lt(y, 4))
  f = guarded(mines, left, down, U32.is_lt(left, 4), U32.is_lt(down, 4))
  g = guarded(mines, x, down, U32.is_lt(x, 4), U32.is_lt(down, 4))
  h = guarded(mines, right, down, U32.is_lt(right, 4), U32.is_lt(down, 4))
  (a + b + c + d + e + f + g + h : U32)

def main() -> U32:
  +mines = Set.add(Set.new(), "1,1")
  a = adjacent(mines, 0, 0)
  b = adjacent(mines, 2, 2)
  (a + b : U32)
```

Listing 13.2: 端の外へ回り込まない 8 近傍計算

コードでは幅と高さを 4 とし、地雷位置を"x,y" の文字列として Set に入れます。guarded は位置が盤面内かどうかを調べてから、地雷の有無を数値にします。Set.has が集合と Bool を返す理由は第 4 章のとおりです。例では (1,1) に一つだけ地雷があり、(0,0) と (2,2) はいずれも周囲に一つ地雷があるため、結果は 2 です。

前巻のライフゲームは盤面の端が反対側へつながる**トーラス境界**でした。しかしここでは端をまたぎません。**同じ「周囲 8 マス」でもゲームの仕様で境界条件が違う**ことを意識しましょう。

13.3 操作を定義する順序

次に定義すべき操作は、旗の切替、マスの公開、地雷を踏んだときの終了、0 のマスの周囲の連鎖公開、勝利判定です。ただし、すべてを一度に実装しません。まず旗の切替のように局所的な操作を `law` で定めて検査します。それから盤面全体へ広げます。

最初のクリックで地雷を踏まない規則を採用するなら、**初手前にはまだ地雷を置かず、初手が決まってからその場所を避けて配置する**という設計も可能です。別の実装方法もありますが、どの方式であっても初手安全の条件を仕様に含める必要があります。

ハンズオン／確認問題

地雷が一つだけ (0,0) にある 4×4 盤面を考えます。(0,1)、(1,1)、(3,3) に表示される数字を予想し、サンプルの座標を変更して検査しましょう。

13.4 16 マスの状態を Array に格納する

前節までは 1 マスの状態と、周囲 8 か所の計算を別々に見てきました。実際のゲームでは 16 マスをまとめ、クリックした位置だけを更新する必要があります。内部の地雷位置は 16 ビットの `U32` で、プレイヤーに見える状態は 16 要素の `Array` で表す設計ができます。

```
type State is Type:
  State{mines: U32, cells: Array<Game.Square>}

def new_state(mines: U32) -> State:
  State{mines, Array.new(Game.Square, 4n, Game.Hidden{})}
```

Listing 13.3: 盤面の内部状態を分けて持つ（参考コードの抜粋）

`State` は `Array` を含むため `Type` に属します。`4n` は「要素数 4」ではなく深さ 4 を表し、$2^4 = 16$ 要素を作ります。表示するマスの状態と内部の地雷配置を一緒に管理していても、**ソルバーへ地雷の正解を渡してよいわけではありません**。運ゲー無し版では、公開情報のみを別の入力型で扱います。

ここで問題になるのが、盤面外のクリックです。`Array` の低水準操作は、ゲームの「範囲外は何もしない」という規則を代わりに保証しません。参照コードの `toggle_checked` と `reveal_checked` は `index < 16` を判定してから更新します。標準 API は利用しつつ、ゲーム固有の条件は明示的に書くことが大切です。

ハンズオン／確認問題

4×4 の盤面では (1,1) が添字 5、(3,3) が添字 15 になります。座標から添字を求める式を説明し、添字 16 の操作を無視すべき理由を考えましょう。

Chapter 14

表示と入力をテストする

マウス座標と盤面座標を対応させ、GUI なしで確かめられる処理を増やす。

14.1　クリックの位置を盤面の番号へ変換する

マウスから得るのは画面上の座標です。しかしゲームの盤面は「上から何行目、左から何列目のマスか」で扱います。ここに変換が必要です。

例えば、32 画素のマスを横に 8 個並べた場合、画面の $x = 64$ は左から 3 列目（0 から数えると 2 列目）です。$y = 32$ なら上から 2 行目（0 から数えると 1 行目）です。盤面を 1 次元の配列で表すならインデックスは

$$\left\lfloor \frac{y}{32} \right\rfloor \cdot 8 + \left\lfloor \frac{x}{32} \right\rfloor$$

となり、この位置は 10 です。

```
import Base

# Translate a mouse position to the index of an 8x8 grid of 32-pixel squares.
# Invalid positions return None rather than selecting an unrelated square.

def cell_if_valid(x: U32, y: U32, valid: Bool) -> Maybe<&2, U32>:
  match valid:
    case False{}:
      None{}
    case True{}:
      Some{U32.add(U32.mul(U32.div(y, 32), 8), U32.div(x, 32))}

def pixel_to_cell(+x: U32, +y: U32) -> Maybe<&2, U32>:
  cell_if_valid(x, y, Bool.and(U32.is_lt(x, 256), U32.is_lt(y, 256)))

def cell_or_zero(result: Maybe<&2, U32>) -> U32:
  match result:
    case Some{cell}:
      cell
    case None{}:
      0

def main() -> U32:
  a = cell_or_zero(pixel_to_cell(64, 32))
  b = cell_or_zero(pixel_to_cell(256, 32))
  (a + b : U32)
```

Listing 14.1: マウスの画素座標を盤面インデックスへ変換する

この例は画面の大きさを 256×256 画素としています。画面の外側 $x = 256$ を、そのまま配列の一つのマスだと思ってはいけません。`pixel_to_cell` は有効な座標なら `Some{index}`、範囲外なら `None{}` を返します。GUI から受け取った値を使う前に**範囲を確かめる**ことが重要です。

14.2　テストを分けて考える

座標変換、周囲の地雷数、旗を付けられるかの判定は、ウィンドウを開かずに実行できます。境界上、範囲外、最初と最後のマスのような例で確認しましょう。

一方、ウィンドウが実際に開くか、クリックイベントが届くか、描画した色や大きさが意図どおりかは、実際の画面が必要です。純粋関数のテストが成功しただけで、GUI の実機動作まで確認できたとは言えません。`Window` の直接操作や `Audio` は独立した API として扱い、基本の `App` を理解してから学びます。

ハンズオン／確認問題

座標 (255,255)、(256,0)、(0,256) を入力した結果を予想しましょう。直前のサンプルの入力値を変え、予想と一致するか確かめてください。

PART IV

仕様駆動開発と形式的証明

この Part の目次

15 law を先に書く 38
16 等式、帰納法、補題と書き換え 41
17 盤面の不変条件 43
18 証明の範囲と信頼境界 46

Chapter 15

law を先に書く

仕様、実装、証明を別々の役割として考え、期待する性質を不足なく述べる。

15.1　まず「こう動くはずだ」を書き出す

旗をクリックで付け外しする関数を作る前に、どう動くべきか考えます。閉じたマスに旗を付けると旗付きになり、旗付きのマスでは旗が外れ、すでに開いたマスの表示は変わりません。これはコードを書く前に決める**仕様**です。

もう一つ、同じマスの旗を二回切り替えると元に戻るはずです。後の確認に使うため、この性質を `law` として書いておきます。

```
import Base

type Cover is Data:
  Hidden{}
  Marked{}
  Open{}

def toggle(c: Cover) -> Cover:
  match c:
    case Hidden{}:
      Marked{}
    case Marked{}:
      Hidden{}
    case Open{}:
      Open{}

law toggle_twice:
  for c: Cover
  {toggle(toggle(c)) == c : Cover}

def toggle_twice(c):
  match c:
    case Hidden{}:
      {==}
    case Marked{}:
      {==}
    case Open{}:
      {==}

law opened_stays_open:
  {toggle(Open{}) == Open{} : Cover}

```

```
33 def opened_stays_open():
34   {==}
```

Listing 15.1: 旗操作の定義、法則、証明

この例の toggle は三つの状態で場合分けします。次の law toggle_twice は、どの Cover でも二回の操作の後は元に戻るという**命題**です。さらに同名の def toggle_twice で、三つの状態に分けた**証明**を記述します。for c: Cover は、任意の Cover 値を対象にする指定です。

15.2 見かけ上もっともらしい仕様の落とし穴

ここで「二回で戻る法則を証明できたから、旗操作は完成だろうか」と考えてみます。実は、**何も変更しない関数**でも二回で元に戻ってしまいます。つまり、この法則は正しいけれど、それだけでは弱すぎます。

そこで「閉じたマスに操作すると旗付きになる」「旗付きでは閉じた状態へ戻る」「開いたマスは変更しない」も別々に主張します。具体的な場合についての小さな law をまず検査し、その後、任意の状態に関する法則へ進むと理解しやすくなります。

22_result_and_game.bend では、開いたマスに旗を付けようとすると Fail になるようにしています。これは toggle とは**異なる操作の仕様**です。二つの関数を混ぜず、どちらの振る舞いを採用するか設計段階で決めることが大切です。

15.3 証明済みと、動作確認済みは違う

```
1 bend examples/04_flag_laws.bend --verdict
```

これは学習用 ZIP を展開したフォルダーで実行するコマンドです。--verdict で法則の検査結果を確認します。返り値の例だけを実行したときの成功と、すべての値に対する法則が証明されたことは同じではありません。また、この章の法則を証明しても、ゲーム全体の勝敗判定や GUI の入力まで正しいとは限りません。

ハンズオン／確認問題

二回の旗切替で元に戻るという法則と、閉じたマスが旗付きになるという法則の違いを、何もしない関数を例に説明してみましょう。

15.4 要求を別ファイルへ分けてから証明する

学習用の小例では、一つの Bend ファイルに実装と法則を書けます。しかしゲームの規則が増えると、「実装を直すついでに仕様を変更してしまう」危険があります。そこでマインスイーパーの参考実装では、要求を LAWS.bend、実装を game.bend など、証明を PROOF.bend に分けています。

```
1 law hidden_is_flagged:
2   {Game.toggle_flag(Game.Hidden{}) == Game.Flag{} : Game.Square}
3
4 law flagged_is_hidden:
5   {Game.toggle_flag(Game.Flag{}) == Game.Hidden{} : Game.Square}
```

Listing 15.2: 実装前に宣言した旗の規則

二回旗を切り替えたら元へ戻るという性質だけでは、何もしない関数も通ってしまいます。だから**隠れたマスは旗付きになり、旗付きのマスは隠れた状態へ戻る**という具体的な要求を別に持たせます。実装が間違っていても、要求を弱めて合格させるのではなく、実装を直します。

参考実装は 31 本の法則を宣言し、同数の証明を `PROOF.bend` に記述しています。現在の読者用 ZIP にはこの全ソースを含めていないため、必要な場合はリポジトリの参考実装から確認してください。教材とサンプルを使うためにリポジトリ全体をクローンする必要はありません。

ハンズオン／確認問題

「旗操作を二回すると元へ戻る」が成立するのに「旗を一回押すと立つ」が成立しない関数を考えてみましょう。後者がない仕様を採用すると、ゲームの操作として何が不足するでしょうか。

Chapter 16

等式、帰納法、補題と書き換え

小さな場合分けから帰納的なデータの証明へ進み、どの部分を証明したか把握する。

16.1　三つの状態の証明から始める

前章の `Cover` は取り得る状態が三種類です。`match` でそれぞれ調べれば、有限個の場合分けで証明できます。`toggle_twice` を読んで、各 `case` でどの式が左右同じになるか考えてみましょう。`{==}` は、その場で確認したい等式を証明するための記法です。

しかし、`List` のように長さがいくつでもあり得る値では、すべての要素数を個別に列挙できません。そこで**数学的帰納法**を使います。

16.2　任意の Bool について法則を証明する

具体例を一つだけ調べるのではなく、**どんな真偽値についても**成り立つ法則を考えます。例えば真偽を二回反転すると元へ戻ります。真と偽の二通りしかないので、それぞれに分ければ確かめられます。

```
import Base

# Unlike a proof for one input, this law covers EVERY Boolean value.
law not_twice:
  for b: Bool
  {Bool.not(Bool.not(b)) == b : Bool}

def not_twice(b):
  match b:
    case True{}:
      {==}
    case False{}:
      {==}

def main() -> U32:
  Bool.to_u32(Bool.not(Bool.not(True{})))
```

Listing 16.1: Bool の全値について証明する

`law` で `for b: Bool` を宣言することで、固定した入力ではなく任意の `Bool` 値について性質を主張します。証明側の `def not_twice` は `True` と `False` の場合をそれぞれ確認します。

```
bend examples/24_bool_proofs.bend --verdict
```

Listing 16.2: 法則の検査

この法則の保証は `Bool.not` と `Bool` に関するものです。マインスイーパーの盤面全体が正しいことを保証するわけではありません。「法則が何について述べているか」を最初に言葉で確認しましょう。

16.3 再帰と帰納法の対応を見る

`List` が空なら性質を確かめられるとします。さらに、残りの部分 `t` について性質が成り立つと仮定したとき、先頭に一要素 `h` を付けた `h <> t` についても成り立つことを示せれば、任意の長さで証明できます。

この考え方は、前巻の `Nat` に関する帰納法の例にも現れます。`Nat` の `On` が基底部、`1n+p` が再帰部です。再帰的な定義と、帰納的な証明の形が似ていることを確認してください。証明の呼び出しが停止しないと、根拠のない命題まで受け入れられてしまうため、終了性の条件は重要です。

16.4 等式を使い回すための操作

証明には一つの計算だけでは足りないことがあります。`Equal.sym` はすでに得た等式の向きを逆にし、`Equal.trans` は二つの等式をつないで新しい等式を導くのに使います。例えば $a = b$ と $b = c$ があれば、$a = c$ と分かります。

複雑な等式を一度に解こうとせず、「まずこの補題を示す」「その補題を別の場所で使う」という分割が読みやすい証明につながります。式変形にどの補題が必要か、紙上で中間式を書いてから証明コードに移すとよいでしょう。

16.5 証明の信頼範囲

`Sigma` や `Exists` などの存在命題では、値の存在と、その値が満たす性質を一緒に表します。ただし、型が通ること、プログラムをいくつか実行して期待値を得ること、`law` の証明が検査されることは、異なる水準の確認です。`@unsafe` や外部効果を含む実装についても、何が保証されたかを個別に検討しなければなりません。

```
bend examples/04_flag_laws.bend --verdict
bend examples/22_result_and_game.bend --verdict
```

`?TODO` などの未完成部分を残したまま、証明が終わったと扱わないでください。

ハンズオン／確認問題

$a = b$ と $b = c$ の等式があるとき、$a = c$ を導く前に、双方の式に現れる中間値を見つけてみましょう。

Chapter 17

盤面の不変条件

マス単位の法則と盤面全体の仕様を区別し、更新前後に何を守りたいか考える。

17.1　一つのマスから盤面全体へ

前章では旗を切り替える処理について、入力したマスの状態がどう変わるかを考えました。では、一つのマスを開いたとき、**ほかのマスや地雷の配置まで変わってよい**でしょうか。通常のマインスイーパーでは、すでに決まった地雷配置は変わらないはずです。

ある操作を行っても保たれる性質を**不変条件（invariant）**と呼びます。例えば、地雷を固定した盤面でマスを開く操作について、次の性質を考えられます。

$$\mathrm{mines}(\mathrm{reveal}(b, p)) = \mathrm{mines}(b).$$

$\mathrm{mines}(b)$ は盤面 b の地雷集合、$\mathrm{reveal}(b, p)$ はマス p を開いた後の盤面を表します。どのマスを開いても地雷の位置が変わらないことを述べています。

17.2　公開したマスは閉じない

もう一つ、すでに公開したマスが次の一手で再び隠れるのはおかしいでしょう。公開済みのマス集合を $\mathrm{open}(b)$ とすると、期待する性質は

$$\mathrm{open}(b) \subseteq \mathrm{open}(\mathrm{reveal}(b, p))$$

です。「後の集合が大きいか等しい」という意味なので、すでに開いたマスをもう一度クリックした場合でも成立します。これを**単調性（monotonicity）**といいます。

ただし、新しいゲームを始めて盤面を初期化するときには、公開済みの集合は空になります。このため、**どの操作に対する不変条件なのか**を明示しなければなりません。初期化とマスを開く操作を一つの法則にまとめると、正しい仕様を書けなくなります。

17.3　0 の連鎖公開と表示する数字

開いたマスの周囲の地雷数が 0 なら、安全な隣接マスも次々に開く処理を考えます。この場合、開くマスは一つとは限りません。しかし公開済み集合は増える一方で、地雷の位置は変わらないという性質は、同じように期待できます。

表示する数字についても、開いたマス p に表示された値が、周囲 8 マスの地雷数に一致することを仕様として定めます。端のマスでは存在しない座標を数えない点も、第 13 章で学んだとおりです。

17.4 証明をどこから始めるか

これらは**これから実装で満たすべき仕様**です。この章の式を読むだけでは、盤面全体が形式的に証明されたことにはなりません。まず「旗の操作」「隣接位置の列挙」「マスを開く操作」のように局所的な処理を分け、その後で盤面全体の更新について証明を考えます。

Bend では `law` で要求を先に記述し、`def` による実装と証明を対応付けます。規模が大きくなれば、要求を `LAWS.bend`、検証する定義を `PROOF.bend` へ分けることもできます。入門段階ではまず小さな法則を一つずつ確認しましょう。

ハンズオン／確認問題

「開いたマスが再び隠れない」という性質について、マスを開く操作とゲームのリセット操作のどちらに適用できるか説明してみましょう。

17.5 不変条件を実際の盤面に結び付ける

盤面を更新しても地雷の位置が変わらないという要求を、単なる日本語の説明で終わらせないようにしましょう。内部状態が `mines` と `cells` の組であるなら、旗操作後の地雷マスクが元と一致することを次のように書けます。

```
law board4_toggle_keeps_all_mines:
  for mines: U32
  for cells: Array<Game.Square>
  for index: U32
  {Board4.mine_mask(Board4.toggle_at(
    Board4.State{mines, cells}, index)) == mines : U32}
```

Listing 17.1: 盤面全体の地雷マスクが不変である

一つの地雷だけを検査しているわけではありません。`for mines: U32` は任意のビットマスク、`for cells` は任意のマス列を対象とします。この法則について `PROOF.bend --verdict` の検査が成功していることと、**盤面のすべてのゲーム進行が正しいこと**は別です。

17.6 クリックの勝敗を状態として表す

左クリックを単なるマスの更新として扱うと、地雷を開いた後に何を返すかが曖昧になります。そこで参考実装では `Playing`、`Won`、`Lost` を区別します。たとえば地雷があるマスをクリックすると敗北することを、実装より先に次の法則で指定します。

```
law session_mine_click_loses:
  {Session.phase_of(Session.click(Session.new(1), 0))
    == Session.Lost{} : Session.Phase}
```

Listing 17.2: 地雷を開いたら敗北

旗付きの地雷をクリックしても敗北しないこと、盤面外のクリックで状態を変えないこと、終了後に再度クリックしても状態が戻らないことも、それぞれ別の法則で表します。勝利条件は「地雷以外のマスがすべて公開されたこと」であり、旗の個数が地雷と同じだけでは不十分です。

勝利判定の参考実装では `Array.clone` と `Array.to_list` を使い、盤面を保持しつつマスを調べています。標準 API が目的に合うなら活用しますが、単純な分岐を不要な抽象化へ置き換える必要はありません。

17.7 証明と独立した実行確認を並べる

現在の開発用 CI では、31 本の法則の形式的証明に加えて、4×4 盤面の近傍関係 256 ケース、数字・添字操作 154 ケース、ゲームの遷移 86 ケースを独立した期待値と照合しています。ただし、一般の地雷配置について数字計算を全称的に証明したわけではありません。また**初手安全・0 の連鎖公開・GUI の実機動作**も未完成です。

ハンズオン／確認問題

「地雷配置が変わらない」と「勝敗判定が正しい」はなぜ別の性質でしょうか。地雷配置を変えずに勝敗だけ間違えるプログラムの例を考え、どの法則が必要か説明してみましょう。

Chapter 18

証明の範囲と信頼境界

形式的証明と実機テストが保証することを区別し、自分のプログラムについて説明できるようになる。

18.1　一つの法則が通れば全部正しいだろうか

旗を二回切り替えると元に戻ることが証明できたとします。このとき、**マウスクリックを正しいマスに変換する処理**まで正しいでしょうか。そうとは限りません。法則の対象は旗を切り替える純粋な関数であり、GUI から座標を取得する過程は別です。

同じことは保存にも当てはまります。保存する文字列を正しく組み立てる純粋関数について法則を証明しても、ディスクがいっぱいで書き込めない場合があります。外部環境に依存する操作を、純粋関数の性質だけでは保証できません。

18.2　検査を三つの段階に分ける

1. **型検査**：引数や返り値、所有権の制約に違反していないかを確かめます。例えば `File` の扱い方が型に合うかを調べます。
2. **実行テスト**：入力を決めて実際の結果を調べます。保存したファイルを読み戻し、文字列が一致するか確かめるのが例です。
3. **形式的証明**：`law` で述べた性質について、指定した前提のもとで一般的に成り立つ根拠を検査します。個々の入力例だけを試すこととは違います。

どれも大切ですが、互いの代わりにはなりません。とくに GUI の画面表示や Audio の音声出力は、実機の確認を別に行う必要があります。

18.3　外部コードや unsafe があるとき

C・JavaScript の外部実装や `@unsafe` を使う場合は、何が純粋な証明の範囲に含まれ、何が外部実装へ任されているかを調べます。`@unsafe` は、終了性などの保証を外せるため、証明の信頼範囲が変わります。型注釈を丁寧に書いただけで外部コードの振る舞いまで正しいとは言えません。

18.4　実際にどのように報告するか

「旗切替関数について、二回実行すると元へ戻る法則を形式的に検査した」「ファイル保存と読み込みを一組として、指定した文字列で実行テストした」「GUI は型検査だけで、画面でのクリック操作は未検証」のように、**主張と検査範囲を対にして記録**しましょう。

「プログラム全体の正しさが証明された」と書くためには、必要な性質が過不足なく挙げられ、それらについて妥当な証明が検査されている必要があります。局所的な法則一つの成功を、ゲーム全体へ広げてはいけません。

ハンズオン／確認問題

「公開するマスの数字が正しい」「保存が毎回成功する」「クリック位置が合っている」の三つについて、純粋関数の形式的証明と実機テストをどう組み合わせるか考えてみましょう。

PART V

並列実行とその他の応用

この Part の目次

19 CPU、GPU、Array.atomic　48
20 C・JavaScript と外部連携　51

Chapter 19

CPU、GPU、Array.atomic

純粋計算の並列呼び出し、IO の並行実行、共有配列の更新を区別し、必要なときだけ正しい機能を選ぶ。

前巻では一つの計算を複数に分ける書き方を学びました。ここでは**何を並列にできるのか**を先に考えます。同じ画面を描くために重い計算を二つ行う場合と、ファイルを待ちながら別の仕事をする場合では、適切な仕組みが違います。

19.1　計算を独立した二つの枝へ分ける

```
import Base

# Independent branches form a balanced fork/join computation.
def count_leaves(+level: Nat) -> U32:
  match level:
    case 0n:
      1
    case 1n+k:
      x y = count_leaves(k) count_leaves(k)
      (x + y : U32)

def main() -> U32:
  count_leaves(8n)
```

Listing 19.1: 均等な二分木での並列呼び出し

このサンプルの中心は次の一行です。

```
x y = count_leaves(k) count_leaves(k)
```

Listing 19.2: 独立した二つの再帰呼び出し

`count_leaves(0n)` は 1 を返し、深さを一つ増やすと左右の結果を足します。したがって深さ n の葉の数を $f(n)$ と書けば、

$$f(0) = 1, \qquad f(n+1) = f(n) + f(n) = 2f(n)$$

となり、`count_leaves(8n)` の結果は 256 です。左右へ同じ深さの仕事を渡しているので、並列にするときの**負荷の偏り**も小さくなります。

二つの呼び出しを同時に始められるからといって、必ず実行時間が半分になるわけではありません。仕事を分配・合流させる費用もあり、短い計算では並列化の方が遅くなる場合があります。

Bend の並列呼び出しにとって大切なのは、**二つの処理が独立していること**と**処理時間がおおむね釣り合うこと**です。後者はプログラムを書く側が判断します。

19.2 GPU 指定の感嘆符は高速化の約束ではない

Bend 2 の公式 GUIDE には、関数名の後ろに! を付ける GPU 側の呼び出し構文があります。例えば、先ほどの関数を `count_leaves!(8n)` と呼ぶ形です。CPU 上の並列実行と GPU 上の実行は同じではありません。GPU がない環境では CPU で処理され、JavaScript ターゲットでは並列呼び出しも逐次実行されます。

GPU が得意なのは、多数の要素へ同じような数値計算を施す処理です。枝ごとの仕事量が異なる探索などでは、分岐の揃わなさが性能上の不利になります。「GPU で実行できること」と「GPU の方が速いこと」を分けて検証してください。また、CPU と GPU での実行差を比較する際には、入力と結果が等しいことを最初に確かめましょう。

このサンプルの CPU での結果 256 は CI で確認していますが、**特定の GPU での速度向上までは測定していません**。性能について報告するなら、実行ターゲット、入力の規模、計測条件を記録する必要があります。

19.3 IO.fork は別の仕事を待ちながら進める

`IO.fork` と先ほどの並列呼び出しは、目的が異なります。例えば外部通信の結果を待つ間に、ほかの IO を進めたい場合です。`IO.fork` は IO の計算を始め、その結果を受け取る `Chan` を返します。`IO.join` はその結果を待ちます。

```
import Base

# fork returns a channel for the result; join waits and closes it.
def main() -> IO(Unit):
  do IO<Unit>:
    chan: Chan(U32) <- IO.fork(U32, IO.pure(U32, 7))
    value: U32 <- IO.join(U32, chan)
    IO.print(U32.show(value))
```

Listing 19.3: IO.fork と IO.join による結果の受け渡し

この例は純粋な値 7 を返す IO を開始して待つので、表示されるのは 7 です。7 を計算するために並列化する利点はありませんが、**返り値がどのように届くか**を理解する練習になります。実際に待ち時間のある通信やファイル処理では、計算の並列化ではなく IO の並行実行が適しています。

19.4 配列を独立に複製するか、共有するか

第 3 章の `Array.clone` は、二つの独立した配列が欲しいときに使いました。一方、`Array.fork` は同じ実体を参照する二つのハンドルを返します。Bend の Base では `Array.fork` と `Array.join` は `@unsafe` の付いた実装であり、共有更新の性質まで純粋な法則だけから主張することはできません。

複数の処理で同じ配列要素を更新すると、読み書きの競合が発生し得ます。そこで共有配列を扱うために `Array.atomic.add` や `Array.atomic.cas` などの**原子的操作**が用意されています。例えば加算の API では、配列と更新前の要素値の組が返ります。どの更新方法を選ぶか、どの型と実行環境で使用可能かは `bend base` と公式 Base で確認してください。

```
bend base Array.atomic.add
bend base Array.atomic.cas
bend base Array.fork
```

Listing 19.4: 共有更新 API の調べ方

マインスイーパーの通常の旗切替や公開操作には、共有配列や原子的更新は必要ありません。一手ずつ純粋関数で盤面を更新すれば、より単純で検証しやすくなります。一方、多数の独立した候補盤面を同時に検査するような処理では、並列計算を検討する価値があります。その際も最初は候補同士の状態を共有しない設計を選び、実際に共有が必要だと分かったときに原子的操作を導入しましょう。

ハンズオン／確認問題

次の三つを分類しましょう。(1) 1 万個の独立した候補盤面の調査、(2) ネットワークの応答を待ちながら画面の更新を進める、(3) 一つの盤面の旗を切り替える。純粋計算の並列化、IO の並行実行、通常の順次更新のどれが自然でしょうか。さらに、共有配列が不要な理由も説明してみましょう。

Chapter 20

C・JavaScript と外部連携

生成するプログラムとホスト環境の境界を知る。

Bend 2 は C のネイティブ実行ターゲットと JavaScript の実行ターゲットを持ちます。GPU やスレッドの振る舞い、使用できる外部機能はターゲット間で同じとは限りません。外部の C・JavaScript 実装を読み込む仕組みを使えば、Bend だけでは用意されていない操作を追加できます。

ただし、外部関数が返す結果や副作用は Bend 内の純粋な証明だけで保証できません。`@unsafe` などの安全性に関する境界とともに理解します。モジュールやパッケージの整理、ライセンス、BendHub 等を使った配布は、まず公式の最新ガイドで対応状況を確かめて進めます。

20.1　作成するファイルの種類を選ぶ

前巻では Bend コードを `bend` で直接実行しました。同じソースから、実行ファイルや C・JavaScript のソースを生成することもできます。**実行する**操作と、**他の処理系で扱うファイルを作る**操作は区別して覚えます。

```
bend examples/15_string_parse.bend
bend examples/15_string_parse.bend -o parse.c
bend examples/15_string_parse.bend -o parse.js
bend examples/15_string_parse.bend -o parse.mjs
```

Listing 20.1: Bend 2 で生成先を選ぶ例

1 行目はコードを実行します。2 行目は C のソース、3 行目は JavaScript のソース、4 行目は JavaScript から読み込める ES モジュールを作ります。`.mjs` へ出力したモジュールからは、Bend の非 IO 関数を利用できます。ただし、外部言語と値を受け渡すときには**コピーされずに同じ配列を参照する**場合があり、所有権の前提を崩さないよう注意が必要です。

20.2　Bend のモジュールとパッケージ

プログラムが大きくなったら、一つのファイルにすべてを書かず、別のファイルを `import ./math.bend as M` のように読み込みます。`M.square(7)` のように、読み込んだモジュールの名前を付けて関数を呼べます。入門編のモジュール例と対応付けて確認しましょう。

公式のパッケージ管理には BendHub があり、内容ハッシュや名前・バージョンでパッケージを参照できます。公開には `--publish` を使用しますが、**公開は取り消せないことがある**ため、練習と

して不用意に公開しないでください。依存するファイルやライセンスも確認したうえで実際の公開を行います。

20.3 生成したコードを動かすまでが検証

C のソースを出力しただけで、対応する実行環境において同じ結果が得られたことにはなりません。JavaScript のモジュールも同様です。例えば次のような確認項目を分けます。

1. Bend 上の実行で既知の入力に対して期待値が得られるか。
2. 生成した C のコードが対象環境のコンパイラでビルドできるか。
3. 生成した JavaScript のコードが対象ランタイムで実行できるか。
4. 入出力、例外、数値型、配列の共有など、環境に依存する振る舞いに違いはないか。

特に JavaScript の外側から Bend の配列を渡すと、値がコピーされず同じ配列を参照する場合があります。Bend 側のアフィン性が、外部言語のコードにも自動的に強制されるわけではありません。配列を別の場所でも保持したい場合は、**誰がそのデータを更新するか**を先に決め、必要なら明示的に複製します。

20.4 パッケージを公開する前に確かめる

開発中は相対パスによる `import` でモジュールを分割し、まずローカルで実行・検査できます。再利用しやすくなった後に、パッケージとして公開するか考えましょう。BendHub へ公開すると、利用者の依存先として参照され得ます。そのため、公開前には依存ファイル、バージョン、ライセンス、再現可能な実行例と検査方法を揃えます。

本書の演習では、公開操作そのものは行いません。`--publish` は外部に成果物を送る操作であり、ローカルのファイル出力と同じ気軽さで試すべきではないからです。公開手順は公式 GUIDE の現行版を確認してください。

20.5 外部実装と形式的証明の境界

Bend の外部効果は、ホスト側の C・JavaScript コードと対応しています。必要な効果を追加することは可能ですが、それらの効果の実行は Bend の純粋な計算に対する証明の範囲外です。`@unsafe` の指定も、停止性などの保証を外す境界になります。外部コードを取り込んだプログラムを「証明済み」と呼ぶときは、**何を証明でき、何を外部環境へ委ねたのか**を明示します。

20.6 Audio は開いてから書く

音を出すための機能も、ファイルと同じように**使い始めと使い終わり**があります。`Audio.open` はサンプルレート（1 秒当たりのサンプル数）を指定してデバイスを開き、`Result` を返します。使い終えたら `Audio.close` で閉じます。

```
import Base

# This must be run with an audio device; CI checks syntax/types only.
def finish(pair: Audio & U32) -> IO(Unit):
  (audio, written) = pair
  Audio.close(audio)
```

```
7
8  def main() -> IO(Unit):
9    do IO<Unit>:
10     audio: Audio <- IO.try(Audio, Audio.open(44100))
11     written: Audio & U32 <- Audio.write(audio, Nil{})
12     finish(written)
```

Listing 20.2: Audio デバイスのオープン・書き込み・クローズ

この例では 44100 Hz の音声デバイスを開き、空のサンプル列を書き込み、閉じます。`Audio.write` は継続して使う `Audio` と書き込んだサンプル数を返します。**空の列なので音は鳴りません**。最初はリソースの順番と型を確認するための例です。音声デバイスがない環境では `Audio.open` が失敗し得ます。

実際に音を鳴らすには、`List<&2,F32>` のサンプル列を用意して渡します。音量、波形、出力デバイスの対応状況も確認する必要があります。音声 API が型検査に通っても、実際にスピーカーから音が鳴ったとは判断できません。

要点

確認できる範囲を区別しましょう。音声、ネットワーク、GUI などの機能では、型検査に成功した後でも外部デバイス・OS・実行ターゲットによって動作が変わります。環境のない自動テストでは、画面表示や音声出力の成功を保証しません。

ハンズオン／確認問題

同じサンプルを C ターゲットと JavaScript ターゲットで動かす際、何を別々に検証する必要があるか列挙してみましょう。

PART VI

マインスイーパー統合と発展

この Part の目次

21 一人で遊べるアプリにする 54
22 発展：運ゲー無しマインスイーパー 58
23 API 索引と次の学習 62

Chapter 21

一人で遊べるアプリにする

> 法則で決めたゲームの規則から始め、初手安全・0 の連鎖公開・GUI・保存を順に結び付ける。

ここまでで、`List` や `Array`、`IO`、`Image`、`App`、`law` といった機能を個別に見てきました。最終的にはそれらを組み合わせ、マインスイーパーを一人で遊べるアプリにします。ただし**個別の関数が正しく動いても、ゲーム全体が完成したとは限りません**。どこまで実装したかを追いながら組み立てます。

21.1　ゲームに必要な状態を先に決める

まず、一つのマスについて画面に見える状態は `Hidden`、`Flag`、`Open{clue}` の三種類です。内部の地雷位置は別に保持します。4×4 の参考実装では、16 ビットの地雷マスクと 16 要素の `Array` を用いる `Board4.State` を採用しました。

クリックを扱うときは、盤面だけでは不十分です。ゲームが続いているか、勝ったか、負けたかも区別します。そこで `session.bend` では次の三種類を使用します。

```
type Phase is Data:
  Playing{}
  Won{}
  Lost{}
```

Listing 21.1: 現在のゲーム進行を表す型

ゲームを構成する要素としては、さらに「地雷数」「公開したマス」「最初のクリックが済んだか」「開始時刻」「表示に使う情報」が必要です。すべてを最初から一つの巨大な関数に押し込まず、**規則を決める純粋関数**と**環境から値を受け取る IO** を分けましょう。

21.2　クリックすると何が起きるかを表にする

左クリックについて、まず次の順序を要求します。

1. クリック位置を盤面座標に変換し、範囲外なら何もしません。
2. 終了状態 `Won`・`Lost` なら、操作を受け付けません。
3. 旗付き・公開済みのマスなら、表示を変更しません。
4. 隠れた地雷なら `Lost` へ移ります。
5. 隠れた安全なマスなら真の隣接地雷数を表示します。
6. 安全なマスがすべて公開されたら `Won` へ移ります。

旗を付け外しする右クリックも、公開済みの数字を書き換えてはいけません。規則の一つひとつを `LAWS.bend` に宣言してから実装することで、後から別の処理を書き加えても仕様が揺れにくくなります。

開発用の参考実装では、例えば `Session.click(Session.new(1), 0)` が `Lost` となることや、旗付きの地雷をクリックしても敗北しないことを法則として定義しました。これらを含む 31 件の法則は `PROOF.bend --verdict` で検査しています。一方、**ゲーム全体の振る舞いを 31 件だけで証明したわけではありません**。この点を意識しながら先へ進みます。

21.3 一手目を安全にする

通常のマインスイーパーを遊ぶとき、一手目で何も分からないまま地雷を踏んでは不公平です。そのため、一手目が選ばれてから、その位置を避けて地雷を配置する方法があります。

例えば 4×4 盤面で地雷を 3 個置く場合は、最初にクリックした添字を記録し、その添字以外の 15 か所から 3 個を選びます。これなら、最初のクリックで踏んだマスは必ず安全です。地雷数が 15 個以下であること、位置が 0〜15 であること、配置に重複がないことなども合わせて確認する必要があります。

この方式はまだ参考実装に組み込まれていません。現状の `Session.new(mines)` は開始時から地雷配置を受け取るため、一手目で地雷を踏む可能性があります。仕様として、少なくとも「初手として選ばれたマスは地雷ではない」を先に法則化してから、配置操作を実装します。

初手だけを安全にすることと、**その後の全手が推測不要であること**も異なります。運ゲー無し版の条件は次章で別に扱います。

21.4 数字が 0 ならどこまで開いてよいか

マスを一つ開いて数字が 0 なら、周囲の 8 か所に地雷はありません。したがって、その周囲の安全なマスを自動公開できます。新しく開いたマスにも数字 0 が現れたら、そのマスの周囲をさらに調べます。これが **0 の連鎖公開**です。

考え方は、これから調べるマスの**作業リスト**を持つことです。

1. 初めにクリックした安全なマスを作業リストへ入れます。
2. 一つ取り出し、すでに公開済みなら重複処理を避けます。
3. 未公開の安全なマスなら、その位置に対応する数字を計算して開きます。
4. 数字が 0 の場合だけ、盤面内の隣接マスを作業リストへ追加します。
5. 作業リストが空になるまで繰り返します。

注意するのは、**同じマスを何度も追加しないこと**と、**旗付きのマスを勝手に公開しないこと**です。作業リストは `List` など標準の構造を使えます。すでに開いたかの判定は、盤面の公開状態と重複管理の方法を見て設計します。新しい汎用グラフ探索ライブラリを自作する必要はありません。

連鎖公開を追加した後も、地雷配置は不変であること、公開済みの数字を上書きしないこと、公開済みのマスは再び隠れないことが必要です。これらを新しい `law` や独立した実行テストに分けて追加します。現時点では **0 の連鎖公開は未実装**です。

21.5 勝敗は「旗の数」で決めない

間違えやすい例として、地雷が 3 個ある盤面に旗を 3 個置くことを考えます。旗の位置が正しいかどうかは、プレイヤーにはまだ分かりません。仮にすべて正しくても、安全なマスが隠れたままなら勝利ではありません。

勝利の条件は**地雷ではないすべてのマスが開かれていること**です。参考実装では、盤面を保持したまま `Array.clone` と `Array.to_list` で表示状態を調べます。既存の標準 API を使う方が自然な場面なので、独自の配列コピー処理を作らずに済みます。

参考実装の `session.bend` では地雷を踏んだら `Lost`、すべての安全なマスを開いたら `Won` となり、どちらも以後のクリックを受け付けません。これは局所的な法則と独立テストで検査していますが、任意の盤面・任意の操作列に関する完全な全称証明ではありません。

21.6 GUI からゲームの規則へ入力を渡す

画面の表示には `App` と `Image` を使えます。第 12 章の `07_gui_hello.bend` は `App.run` の基本形を示し、第 14 章の `23_cell_hit_test.bend` は画素座標を配列の添字へ変換します。

ただし第 14 章のサンプルは **8×8 盤面・32 画素のマス**を想定しています。4×4 の参考エンジンに接続するなら、幅と高さを 128 画素にし、`index = 4 * row + column` として整合させる必要があります。サンプルごとに盤面の寸法が違うのに、そのままつないでしまうと別のマスが開かれます。

`App` のイベント処理からは、座標と「左クリックか右クリックか」をゲームの純粋関数へ渡す設計にします。ゲーム側から画面用の色や座標を決定しない方が、GUI を起動しなくても規則をテストできます。実際にクリックが届くか、右クリックイベントが OS でどう扱われるか、描画が意図した位置に出るかは、**ウィンドウのある実機で別途確認**する必要があります。

21.7 時間・リセット・保存は境界の仕事

経過時間は現在時刻を読む IO と、時刻の差を計算する純粋関数に分けます。ゲーム終了時に時間を止め、リセット時に新しい開始時刻を設定します。リセットは公開済みのマスを空に戻す操作なので、「クリックで公開済み集合が減らない」という不変条件を無条件に適用してはいけません。

保存は、まず盤面をシリアライズする純粋関数を考え、その文字列を `File.write` で書き込みます。読み戻すときは `Result` で失敗を扱い、盤面サイズ・地雷数・配列の長さ・公開数字などの妥当性を検査します。ファイルが存在するだけで正しい盤面が入っているとは限りません。書き込みに失敗したときも、ゲーム自体を続けられるかどうかを設計します。

21.8 部品から完成へ進むための検査順

ここまでの参考実装には、旗・クリック・敗北・勝利の純粋な部分と、4×4 の近傍判定があります。通常版として必要な初手安全、0 の連鎖公開、完全な GUI・保存の統合は残っています。実装の進捗と、読者が遊べる完成品を混同しないでください。

まず 4×4 盤面の全 256 座標対で近傍関係を調べ、次に地雷マスクと数字の計算、複数クリックでの勝敗を検査します。このように**正解を別の方法で計算するテスト**を使うと、実装とテストが同

じ誤りを含む危険を減らせます。それから初手安全・連鎖公開のテストを追加し、最後に実機の画面操作を確認します。

ハンズオン／確認問題

地雷が 1 個ある 4×4 盤面で、(1) 地雷に旗を立ててから左クリック、(2) 最後の安全なマスを公開、(3) 公開済みの数字を再クリック、(4) 盤面外をクリック、の四つについて期待する結果を説明しましょう。続いて初手安全と 0 の連鎖公開に必要な法則を一つずつ日本語で書き、どの `Array`・`List` の操作を使うと素直か考えてみましょう。

Chapter 22

発展：運ゲー無しマインスイーパー

「公開情報から確実に分かること」を形式化し、ソルバーによる盤面検証と生成を理解する。

22.1　なぜ通常の盤面では推測が必要になるのか

通常のマインスイーパーでは、開いたマスに書かれた数字が同じでも、隠れている地雷の置き方が複数あり得ます。例えば、数字の 1 の周囲に未確定のマスが二つ残り、そのどちらにも地雷が置ける場合、片方を適当に開くことは推論ではありません。

運ゲー無し版では、**最初に開くマスが決まったあと、確実に安全と判定できる手だけを続けて勝利できる盤面**を採用します。Simon Tatham の『Mines』は、この考え方を使った公開実装です。実際にソルバーを用意し、候補の地雷配置を解かせて検証する方法が知られています。

22.2　最小の推論規則を Bend 2 で書く

開かれたマスに表示された数字を c、周囲で地雷と**確定した**マス数を f、まだ確定していないマス数を u とします。

$$r = c - f$$

は、未確定マスに残っている地雷数です。$0 \le r \le u$ が満たされるとき、次の二つの規則が使えます。

- $r = 0$：周囲の未確定マスはすべて安全です。
- $r = u$：周囲の未確定マスはすべて地雷です。

$0 < r < u$ であれば、その数字だけでは決まりません。$r > u$ は矛盾です。プレイヤーが旗を置いたマスを無条件で「確定した地雷」にしてはいけません。誤った旗は誤推論を引き起こすためです。

```
import Base

# Deduce from ONE visible number:
# remaining = displayed clue - mines already established as certain
# unknown   = number of still-unclassified adjacent squares
#
# "Flagged by the player" is NOT automatically "certain mine".

type LocalRule is Data:
  AllSafe{}
  AllMines{}
```

```
  Unknown{}
  Inconsistent{}

# Definition order matters: each helper is available to those below it.
# Computed conditions are passed as function parameters to pattern matches.
def infer_equal(remaining: U32, unknown: U32, equal: Bool) -> LocalRule:
  match equal:
    case True{}:
      AllMines{}
    case False{}:
      Unknown{}

def infer_zero(+remaining: U32, +unknown: U32, empty: Bool) -> LocalRule:
  match empty:
    case True{}:
      AllSafe{}
    case False{}:
      infer_equal(remaining, unknown, U32.is_eq(remaining, unknown))

def infer_limit(+remaining: U32, +unknown: U32, over: Bool) -> LocalRule:
  match over:
    case True{}:
      Inconsistent{}
    case False{}:
      infer_zero(remaining, unknown, U32.is_eq(remaining, 0))

def infer(+remaining: U32, +unknown: U32) -> LocalRule:
  infer_limit(remaining, unknown, U32.is_lt(unknown, remaining))

# A small observable numeric output for automated testing.
def tag(r: LocalRule) -> U32:
  match r:
    case AllSafe{}:
      1
    case AllMines{}:
      2
    case Unknown{}:
      3
    case Inconsistent{}:
      4

law zero_remaining_is_safe_for_three:
  {infer(0, 3) == AllSafe{} : LocalRule}

def zero_remaining_is_safe_for_three():
  {==}

def main() -> U32:
  a = tag(infer(0, 3))
  b = tag(infer(3, 3))
  c = tag(infer(1, 3))
  d = tag(infer(4, 2))
  (a + b + c + d : U32)
```

Listing 22.1: 局所制約の判定と法則

`infer` は残り地雷数と未確定マス数から結果を表す ADT を返します。`AllSafe` や `AllMines` は推論の種類を表し、マスの座標自体を返すものではありません。`Unknown` の場合に勝手に安全マスを決めてはいけません。入力に矛盾があるときは `Inconsistent` で区別します。付属の `law` は、残り地雷数が 0 で未確定マスが 3 個ある場合の具体例です。任意のマス数や盤面全体を証明したものではありません。

ハンズオン／確認問題

未確定マスが 4 個、残り地雷数が 0、2、4、5 の場合に、それぞれ `infer` が何を返すべきか調べてみましょう。

22.3 複数の数字から推論する

一つの数字だけで決まらない場合でも、複数の数字が制約する未確定マスの集合を比較すると、情報が得られることがあります。集合 A に地雷が a 個、集合 B に地雷が b 個という制約があり、$A \subseteq B$ なら、

$$\#\big((B \setminus A) \cap M\big) = b - a$$

です。特に $b = a$ なら $B \setminus A$ のマスはすべて安全です。$b - a = |B \setminus A|$ ならすべて地雷です。この**部分集合の規則**は、`Set` の活用例でもあります。

さらに、残り地雷の総数を使った推論もあります。独立した場所に分かれる制約でも、総地雷数によって片方の候補が排除されることがあります。標準ライブラリの集合・リスト・辞書をどう使うか検討するのが本書の目的です。

22.4 公開情報と真の地雷配置を分離する

公開状態 V から矛盾なく置ける地雷配置の集合を $\mathcal{C}(V)$ とします。ここには**開いたマスとその数字、盤面サイズ、総地雷数**だけが反映されます。未公開の本当の地雷位置は渡しません。

$\mathcal{C}(V) \neq \varnothing$ であり、未公開マス p がすべての候補で地雷でないなら、

$$\forall M \in \mathcal{C}(V), \quad p \notin M,$$

p は確実に安全です。候補集合が空の場合は、入力が矛盾しているものとしてエラーにします。空集合に対する全称命題を利用して「安全」と判定するのは誤りです。

この区別をコードで守ることが重要です。

- **ソルバー**：公開状態 V だけを受け取り、確実な安全マスを返す。
- **ゲーム本体**：真の配置を保持し、実際に開いたマスの数字を計算する。
- **検証器**：ソルバーの提案した操作をゲーム本体で再生し、最後まで安全にクリアできるか検査する。

22.5 候補生成と検証を繰り返す

初手 p_0 を決め、そのマスを必ず安全にするよう候補の地雷配置を作ります。より大きく開ける開始局面にしたい場合は、初手とその周囲 8 マスを地雷禁止領域にできます。その後、ソルバーに**公開状態だけ**を渡して確実な操作を繰り返させます。

1. 初手が安全な地雷配置を生成します。
2. 初手を開き、数字が 0 なら周囲を連鎖的に開きます。
3. 公開数字と総地雷数から、確実な安全マスを探します。
4. 安全マスが見つかれば開き、全安全マスを開けるまで続けます。
5. 候補が残っているのに確実な手がなければ、その盤面は採用しません。

生成の試行回数には上限を設けます。上限に達した場合は**生成失敗を明示**し、推測が必要な盤面を無断で運ゲー無し版として公開してはいけません。あらかじめ検証した盤面を別途用意して代替する方法もあります。

22.6 小盤面用の参考実装と検証

この節の発展用参考実装はリポジトリの 本書の参考実装にあります。学習用 ZIP には完成コードを入れず、読者が必要に応じて参照できます。

小盤面用の Python 検証器は、公開数字と総地雷数に矛盾しない候補配置を**全列挙**し、そのすべてで安全なマスだけを開きます。列挙数が上限を超えた場合は採用を保留します。これにより、限定した盤面サイズでは、局所規則で分からない確定手まで探せます。

ただし、候補の全列挙は盤面が大きくなると指数的に重くなります。これは本書の学習・テスト用の独立した参考実装であり、実用サイズの Bend 版ソルバーや GUI アプリの完成を意味しません。Bend への移植を進める際は、まず局所推論と集合の差分から取り組みましょう。

22.7 「運ゲー無し」の保証範囲

解が一意であることと、途中の局面で確実な手を選べることは異なります。また、局所規則しか知らないソルバーが手詰まりになっても、もっと強い推論で解ける可能性があります。したがって、ソルバーが最後まで解けた盤面だけを採用する方法は確実ですが、採用されなかった盤面が必ず運ゲーであるとは断言できません。

この教材では「**固定した初手から、このソルバーが公開情報のみで全安全マスを開ける**」ことを明示的な合格条件にします。初手が変わっても常に同じ条件を満たす、すべての推論規則を網羅している、すべての盤面を等確率で生成する、といった強い条件は主張しません。

参考資料

- Simon Tatham (2019), Writing a soluble-grid generator for Mines：候補の改変とソルバー検査。
- JSMinesweeper：推測不要盤面の生成、ソルバー、分析。
- Mindsweeper：公開されている別の無推測盤面生成の実装。

ハンズオン／確認問題

公開状態と真の地雷配置を別の型で定義してみましょう。また、局所推論しか使わないソルバーが行き詰まるが、集合の差分を使えば先に進める例を考えてみましょう。

Chapter 23

API 索引と次の学習

使い方が分からない機能を自分で調べ、検証できるようにする。

標準ライブラリは機能名を列挙するだけでは使いこなせません。必要な操作について**引数・戻り値・所有権・失敗・対象ターゲット**を調べる方法を繰り返しましょう。

分類	主な名前	確認すること
数値	Nat, U32, F32, Word	変換と丸め、上限
文字	Char, String	分割、解析、文字境界
列・配列	List, Array	走査、複製、所有権
辞書・集合	Map, Set	文字列キー、検索結果
結果型	Maybe, Result, Either	失敗の情報
入出力	IO, File, Process	ハンドル、終了コード
通信	TCP, UDP, Socket	接続と切断
並行処理	Chan, IO.fork, IO.join	同期とタイムアウト
GUI	Image, App, Window, Event	画面・イベントの環境
音声	Audio	デバイスと書込み
証明	law, Equal, Sigma	仮定と結論、カーネル

公式資料：Bend 2 公式ガイド、公式 Base ライブラリ。

ハンズオン／確認問題

初めて見る `Base` の関数を一つ選び、最小サンプルを作って型検査し、成功時と失敗時の振る舞いを確かめましょう。